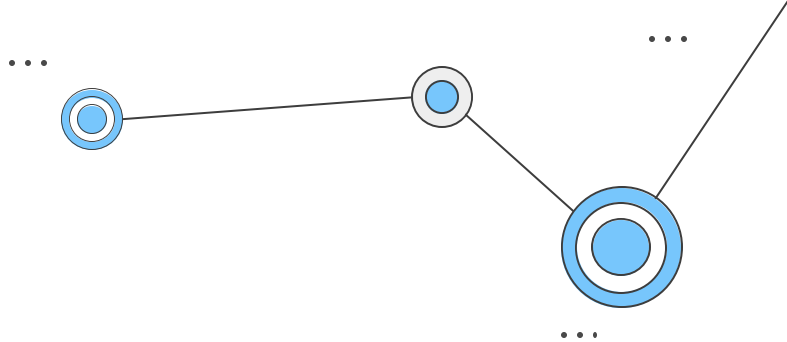
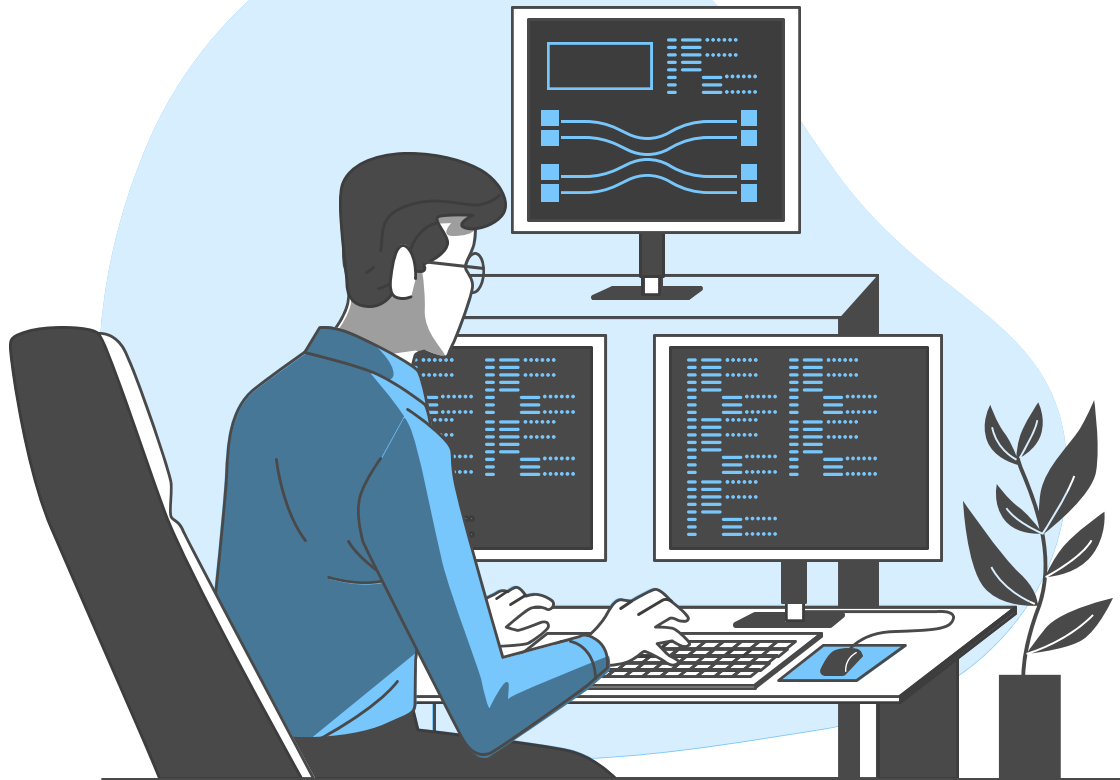




PUC Minas



Laboratório de Desenvolvimento de Software

Prof. Dr. João Paulo Aramuni

Sumário

- Persistência de dados
 - Banco de dados NoSQL
 - Gravação e listagem de dados
 - Atualização e exclusão de dados

Persistência de dados

O que é um Banco de Dados?

Definição da Oracle:

Um **banco de dados** é uma coleção organizada de informações - ou dados - estruturadas, normalmente armazenadas eletronicamente em um sistema de computador. Um banco de dados é geralmente controlado por um sistema de gerenciamento de banco de dados (SGBD). Juntos, os dados e o SGBD, juntamente com os aplicativos associados a eles, são chamados de sistema de banco de dados, geralmente abreviados para apenas BD.

The Oracle logo, consisting of the word "ORACLE" in white, uppercase, sans-serif font, followed by a registered trademark symbol (®), set against a solid red rectangular background.

Persistência de dados

O que é um Banco de Dados?

Definição da Oracle:

Os **dados** nos tipos mais comuns de bancos de dados em operação atualmente são modelados em linhas e colunas em uma série de tabelas para tornar o processamento e a consulta de dados eficientes. Os dados podem ser facilmente acessados, gerenciados, modificados, atualizados, controlados e organizados. A maioria dos bancos de dados usa a linguagem de consulta estruturada (SQL) para escrever e consultar dados.

The Oracle logo, consisting of the word "ORACLE" in white, uppercase, sans-serif font, set against a solid red rectangular background.

Persistência de dados

Os bancos de dados evoluíram muito desde a sua criação no início dos anos **1960**.

Nos anos **1980**, bancos de dados **relacionais** tornaram-se populares, seguidos por bancos de dados orientados a objetos na década de **1990**.

Mais recentemente, bancos de dados **NoSQL** surgiram como uma resposta ao crescimento da internet e à necessidade de maior velocidade e processamento de dados não estruturados.

Hoje, BDs na **nuvem** e BDs autônomos estão abrindo novos caminhos quando se trata de como os dados são coletados, armazenados, gerenciados e utilizados.

Sumário

- Persistência de dados
 - **Banco de dados NoSQL**
 - Gravação e listagem de dados
 - Atualização e exclusão de dados

Persistência de dados

Tipos de BDs:

- Bancos de dados relacionais
- Bancos de dados NoSQL (não relacional)
- Bancos de dados orientados a objetos
- Bancos de dados distribuídos
- Data warehouses
- Bancos de dados gráficos
- Bancos de dados de código aberto
- Bancos de dados em nuvem
- Banco de dados multimodelo
- Banco de dados de documentos/JSON
- Bancos de dados autônomos



Persistência de dados

Tipos de BDs:

- Bancos de dados relacionais
- Bancos de dados NoSQL (não relacional)
- Bancos de dados orientados a objetos
- Bancos de dados distribuídos
- Data warehouses
- Bancos de dados gráficos
- Bancos de dados de código aberto
- Bancos de dados em nuvem
- Banco de dados multimodelo
- Banco de dados de documentos/JSON
- Bancos de dados autônomos

Mais importantes para ingressar
no mercado de desenvolvimento



Persistência de dados

Tipos de BDs:

- **Bancos de dados relacionais**
- Bancos de dados NoSQL (não relacional)
- Bancos de dados orientados a objetos
- Bancos de dados distribuídos
- Data warehouses
- Bancos de dados gráficos
- Bancos de dados de código aberto
- Bancos de dados em nuvem
- Banco de dados multimodelo
- Banco de dados de documentos/JSON
- Bancos de dados autônomos

Foco da disciplina de BD



Persistência de dados

Tipos de BDs:

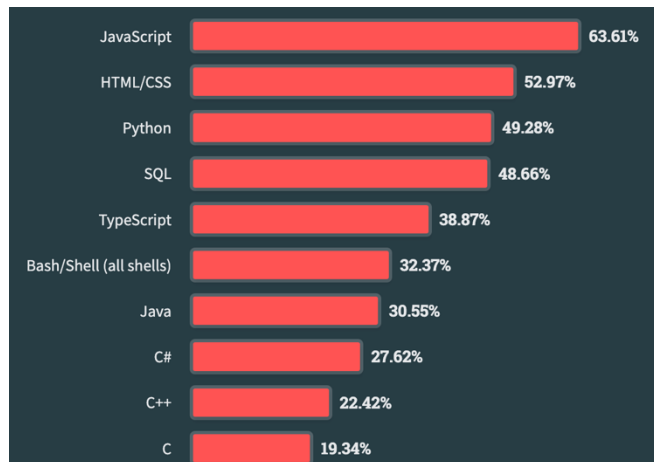
- Bancos de dados relacionais
- **Bancos de dados NoSQL (não relacional)**
- Bancos de dados orientados a objetos
- Bancos de dados distribuídos
- Data warehouses
- Bancos de dados gráficos
- Bancos de dados de código aberto
- Bancos de dados em nuvem
- Banco de dados multimodelo
- Banco de dados de documentos/JSON
- Bancos de dados autônomos

Parte da disciplina de Arquitetura
de Aplicações Web



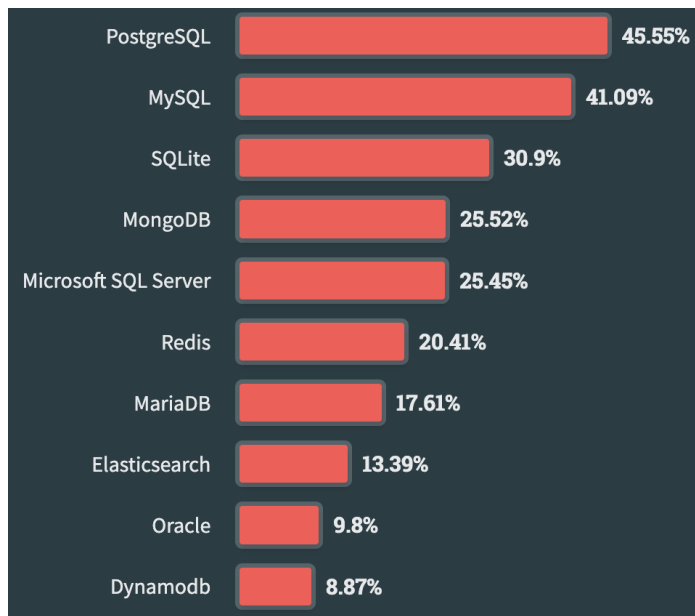
Persistência de dados

- Em maio de 2023, mais de 90.000 desenvolvedores responderam à pesquisa anual do **Stack Overflow** sobre como aprendem e sobem de nível, quais ferramentas estão usando e quais desejam. SQL aparece em 4º no ranking das tecnologias mais populares.
- Outra fonte da JetBrains de 2022:
 - <https://www.jetbrains.com/lp/devecosystem-2022/databases/>



Persistência de dados

- Top 10 BDs mais utilizados segundo a pesquisa do **Stack Overflow** de 2023:



Persistência de dados

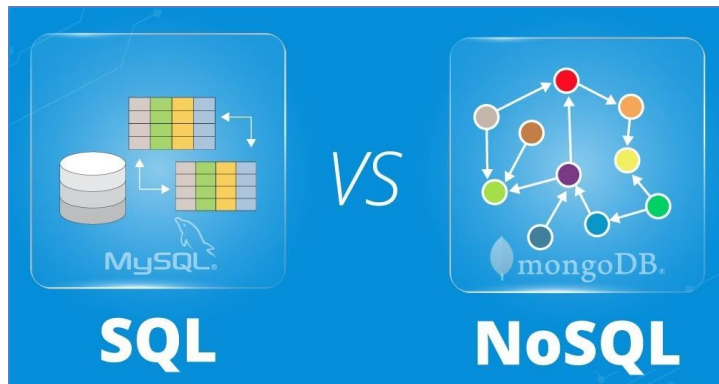
- O **DB-Engines Ranking** classifica os sistemas de gerenciamento de banco de dados de acordo com sua popularidade. O ranking é atualizado mensalmente.

419 systems in ranking, July 2023									
Rank			DBMS	Database Model	Score				
Jul 2023	Jun 2023	Jul 2022			Jul 2023	Jun 2023	Jul 2022		
1.	1.	1.	Oracle 🟡	Relational, Multi-model ⓘ	1256.01	+24.54	-24.28		
2.	2.	2.	MySQL 🟡	Relational, Multi-model ⓘ	1150.35	-13.59	-44.53		
3.	3.	3.	Microsoft SQL Server 🟡	Relational, Multi-model ⓘ	921.60	-8.47	-20.53		
4.	4.	4.	PostgreSQL 🟡	Relational, Multi-model ⓘ	617.83	+5.01	+1.96		
5.	5.	5.	MongoDB 🟡	Document, Multi-model ⓘ	435.49	+10.13	-37.49		
6.	6.	6.	Redis 🟡	Key-value, Multi-model ⓘ	163.76	-3.59	-9.86		
7.	7.	7.	IBM Db2	Relational, Multi-model ⓘ	139.81	-5.07	-21.40		
8.	8.	8.	Elasticsearch	Search engine, Multi-model ⓘ	139.59	-4.16	-14.74		
9.	9.	9.	Microsoft Access	Relational	130.72	-3.73	-14.37		
10.	10.	10.	SQLite 🟡	Relational	130.20	-1.02	-6.48		
11.	11.	📈 13.	Snowflake 🟡	Relational	117.69	+3.55	+18.53		
12.	12.	📉 11.	Cassandra 🟡	Wide column	106.53	-2.03	-7.88		
13.	13.	📉 12.	MariaDB 🟡	Relational, Multi-model ⓘ	96.10	-1.21	-16.42		
14.	14.	14.	Splunk	Search engine	87.12	-2.34	-11.09		
15.	📈 16.	15.	Microsoft Azure SQL Database	Relational, Multi-model ⓘ	78.96	-0.01	-5.94		
16.	📉 15.	16.	Amazon DynamoDB 🟡	Multi-model ⓘ	78.81	-1.10	-5.13		
17.	17.	17.	Hive	Relational	72.87	-2.65	-6.61		
18.	18.	📈 22.	Databricks	Multi-model ⓘ	68.47	+2.65	+17.25		
19.	19.	📉 18.	Teradata	Relational, Multi-model ⓘ	60.25	-2.39	-10.67		
20.	20.	📈 24.	Google BigQuery 🟡	Relational	55.42	+0.78	+6.53		

Persistência de dados

- **SQL vs NoSQL**

- SQL (Structured Query Language) e NoSQL (Not Only SQL) são duas abordagens distintas para armazenar e recuperar dados.
- Pode-se dizer que são dois tipos de BDs. Podemos diferenciá-las em termos de:
I) Modelo de Dados II) Esquemas III) Consultas IV) Escalabilidade e V) Transações.



Persistência de dados

I) Modelo de Dados

- **SQL:** Os SGBDs SQL seguem o modelo de dados **relacional**, onde os dados são organizados em tabelas com linhas e colunas. Cada tabela tem uma estrutura definida por um esquema que especifica os tipos de dados e as restrições dos atributos.
- **NoSQL:** Os SGBDs NoSQL, ou **não relacionais**, têm modelos de dados mais flexíveis e não seguem a abordagem tradicional de tabelas. Existem vários modelos NoSQL, como documentos (exemplo: MongoDB), chave-valor (exemplo: Redis), colunas largas (exemplo: Apache Cassandra) e grafos (exemplo: Neo4j), que permitem que os dados sejam armazenados e organizados de maneiras diversas.

Persistência de dados

II) Esquemas

- **SQL:** Os SGBDs SQL utilizam esquemas rígidos, o que significa que a estrutura das tabelas e os tipos de dados devem ser definidos previamente antes de inserir os dados. Mudanças no esquema podem ser complexas e requerem migrações cuidadosas.
- **NoSQL:** Os SGBDs NoSQL normalmente adotam esquemas flexíveis ou schemaless, o que permite que os dados sejam inseridos sem a necessidade de um esquema pré-definido. Essa flexibilidade permite uma maior adaptação a mudanças nos requisitos dos dados.

Persistência de dados

III) Consultas

- **SQL:** Para recuperar dados de bancos de dados SQL, são utilizadas consultas estruturadas em SQL. As consultas seguem uma sintaxe padronizada e são usadas para recuperar informações de tabelas ou realizar operações de junção e agregação.
- **NoSQL:** As consultas em bancos de dados NoSQL variam de acordo com o modelo de dados. Alguns SGBDs NoSQL utilizam linguagens específicas para acessar os dados, enquanto outros podem oferecer interfaces de consulta mais flexíveis.

Persistência de dados

IV) Escalabilidade

- **SQL:** Os bancos de dados SQL são conhecidos por sua consistência e conformidade ACID (Atomicidade, Consistência, Isolamento e Durabilidade), o que pode limitar a escalabilidade horizontal em ambientes distribuídos.
- **NoSQL:** Os bancos de dados NoSQL foram projetados com foco na escalabilidade e, muitas vezes, adotam modelos BASE (Basicamente Disponível, Soft State e Eventualmente consistente), o que permite uma maior escalabilidade horizontal para lidar com grandes volumes de dados e cargas de trabalho.

Persistência de dados

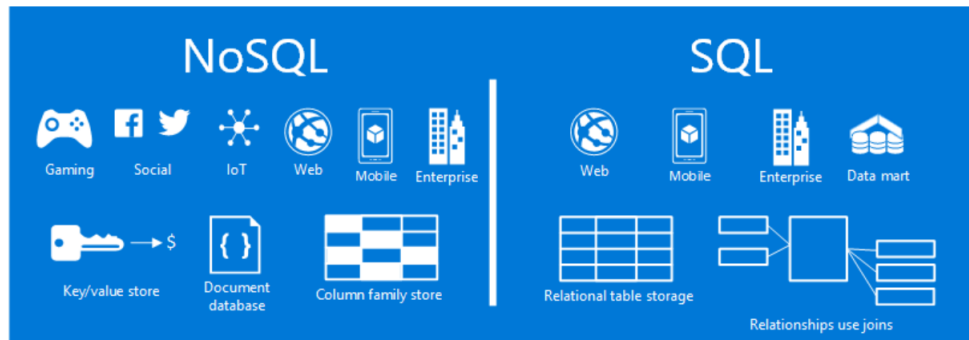
V) Transações

- **SQL:** Os SGBDs SQL oferecem suporte robusto a transações e garantem a consistência dos dados com base no conceito ACID.
- **NoSQL:** Alguns SGBDs NoSQL suportam transações, mas geralmente com uma abordagem mais flexível, permitindo uma consistência eventual em situações de alta concorrência.

Persistência de dados

- SQL vs NoSQL

- Em resumo, SQL e NoSQL são duas abordagens distintas para o gerenciamento de banco de dados, cada uma com suas vantagens e desvantagens.
- A escolha entre SQL e NoSQL dependerá das necessidades específicas do projeto, como modelo de dados, requisitos de escalabilidade, flexibilidade do esquema e tipo de consulta requerida.



Persistência de dados

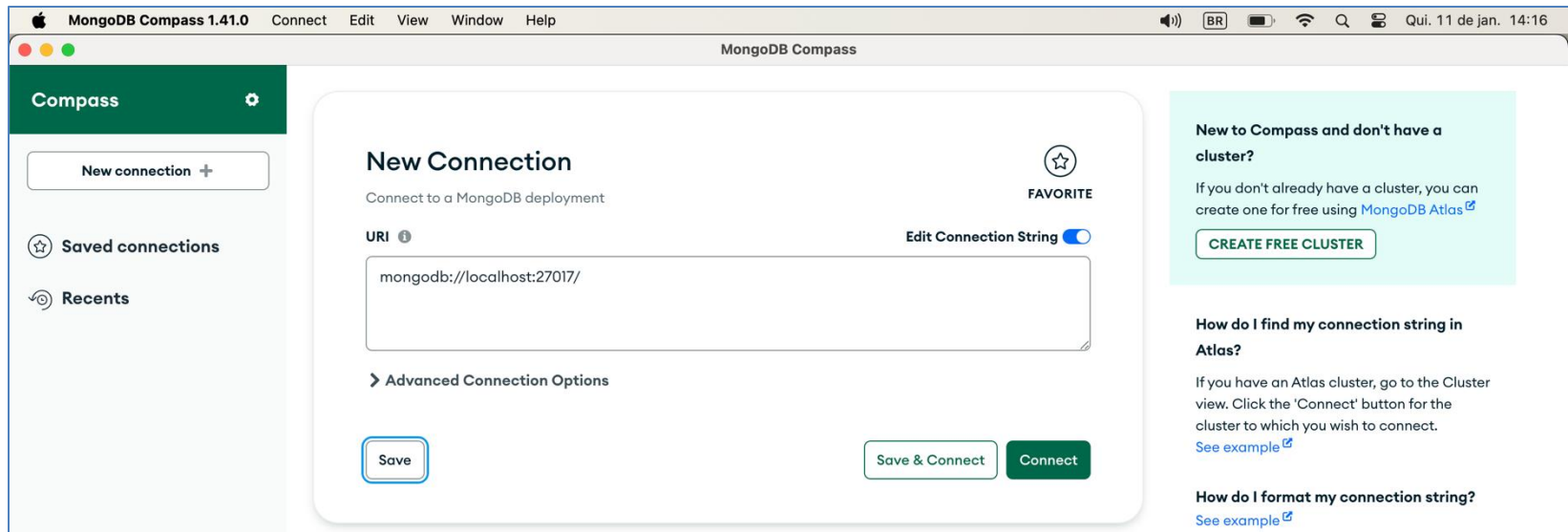
- Nesta disciplina, de **Arquitetura de Aplicações Web**, utilizaremos o **MongoDB**.
 - Trata-se de um Banco de Dados **não relacional**, **NoSQL**.



- Lembrando que, na disciplina de **BD**, utilizamos o modelo **relacional**, com **SQL**.



- **MongoDB**
 - O MongoDB possui um sistema gerenciador de banco de dados (SGBD) oficial, com interface gráfica. O **MongoDB Compass**:
 - <https://www.mongodb.com/try/download/compass>

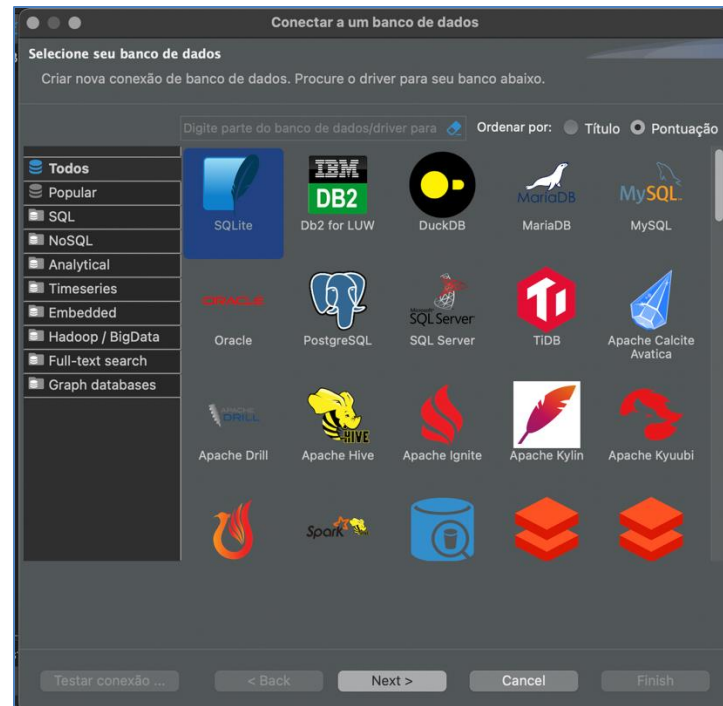


Persistência de dados

- Dbeaver

- O Dbeaver, utilizado na disciplina de BD, só possui suporte para o MongoDB em sua versão Enterprise (paga). Porém, ele oferece 14 dias para teste.

- <https://dbeaver.com/download/enterprise/>



Sumário

- Persistência de dados
 - Banco de dados NoSQL
 - **Gravação e listagem de dados**
 - Atualização e exclusão de dados

Persistência de dados

- **Gravação e listagem de dados**
 - Antes da prática, é necessário entender o que são os **métodos HTTP**.
 - Os métodos HTTP (Hypertext Transfer Protocol) definem as operações que podem ser realizadas em recursos identificados por URLs. Cada método tem um significado específico e é utilizado para realizar diferentes operações no servidor.

Persistência de dados

- **Gravação e listagem de dados**
- Aqui estão alguns dos principais métodos HTTP:
 - **GET:** Solicita a representação de um recurso específico. Não deve ter efeitos colaterais no servidor.
 - **Exemplo:** Obter informações de um usuário - **GET /users/123**
 - **POST:** Envia dados para serem processados para um recurso específico. Pode resultar na criação de um novo recurso ou em alterações no estado do recurso.
 - **Exemplo:** Criar um novo usuário - **POST /users**

Persistência de dados

- Gravação e listagem de dados
 - Aqui estão alguns dos principais métodos HTTP:
 - **PUT**: Atualiza completamente um recurso existente ou cria um novo recurso se ele não existir. Substitui completamente o estado do recurso.
 - **Exemplo**: Atualizar informações de um usuário - **PUT /users/123**
 - **DELETE**: Remove um recurso específico do servidor.
 - **Exemplo**: Excluir um usuário - **DELETE /users/123**

Persistência de dados

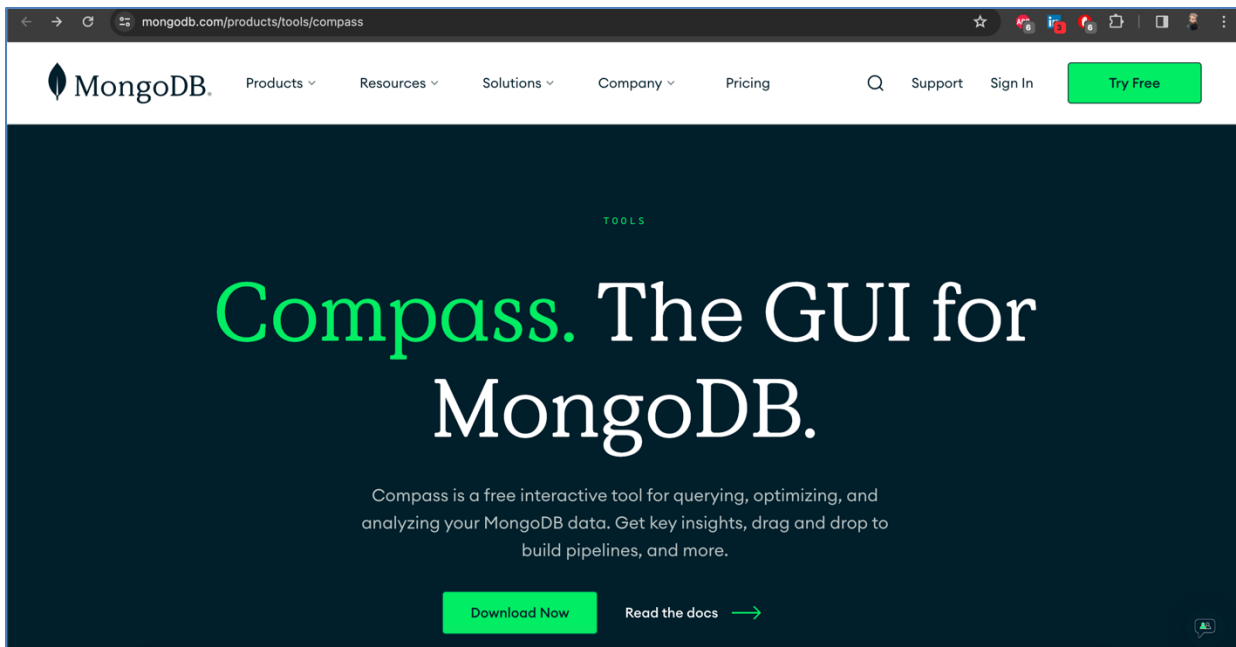
- **Gravação e listagem de dados**
 - Agora vamos criar uma **API REST** para gravar, listar, atualizar e excluir um usuário do banco de dados.
 - É comum chamarmos esse tipo de sistema de CRUD.
 - CRUD é um acrônimo que representa as quatro operações básicas realizadas em sistemas que envolvem manipulação de dados e persistência.
 - As letras em CRUD correspondem a: **C**reate, **R**ead, **U**ppdate e **D**eleite.

- **Gravação e listagem de dados**

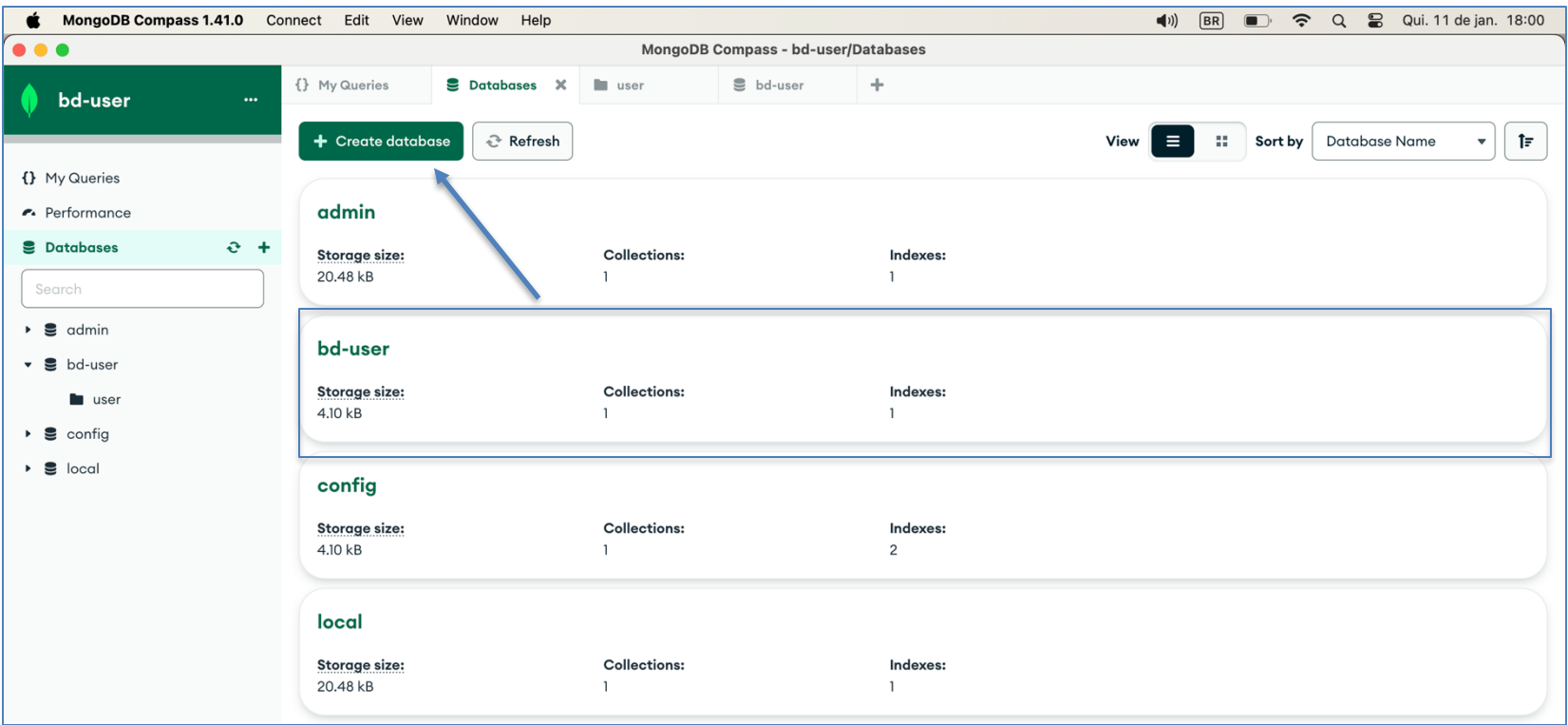
- Para iniciar a prática, é necessário primeiro criar o banco de dados

MongoDB. Vamos utilizar o **MongoDB Compass**:

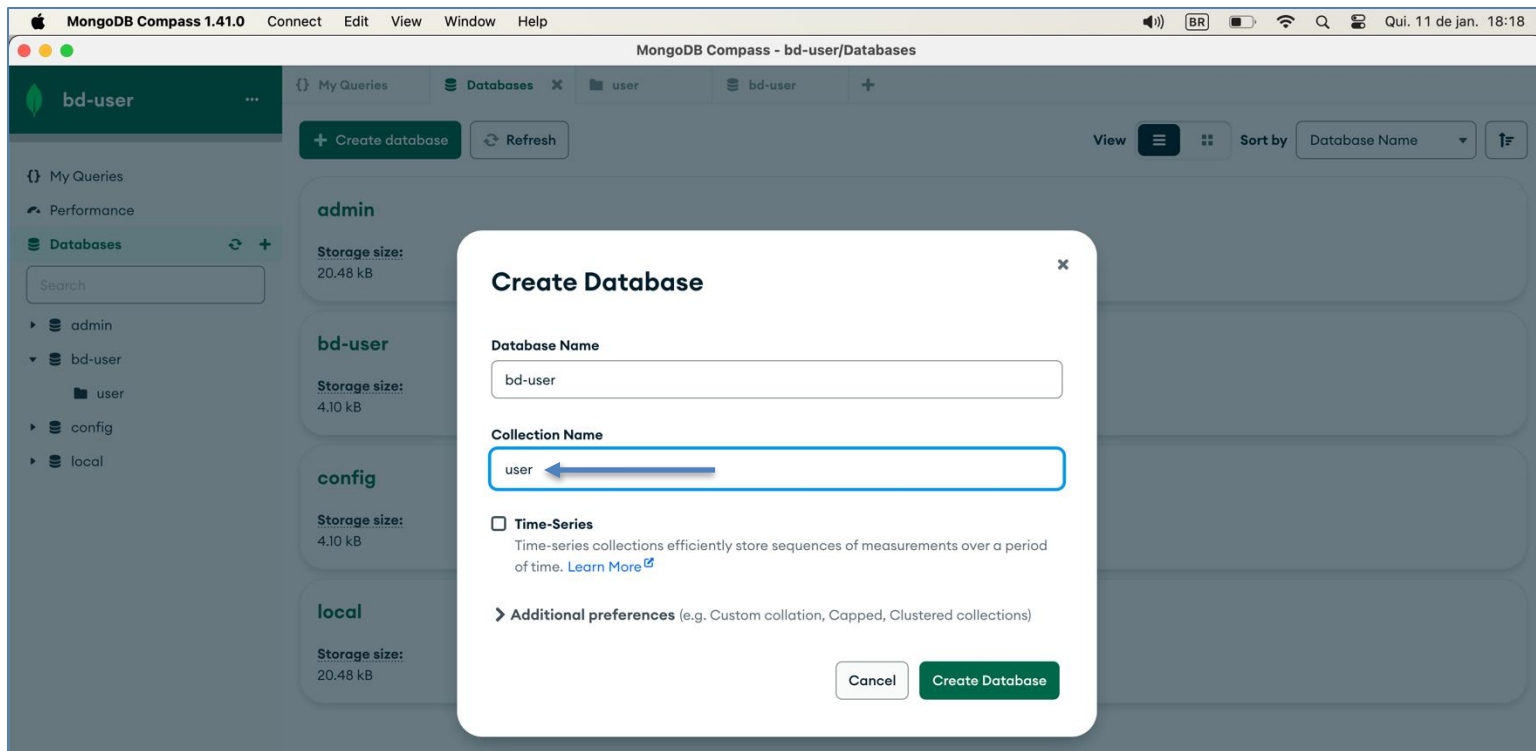
- <https://www.mongodb.com/products/tools/compass>



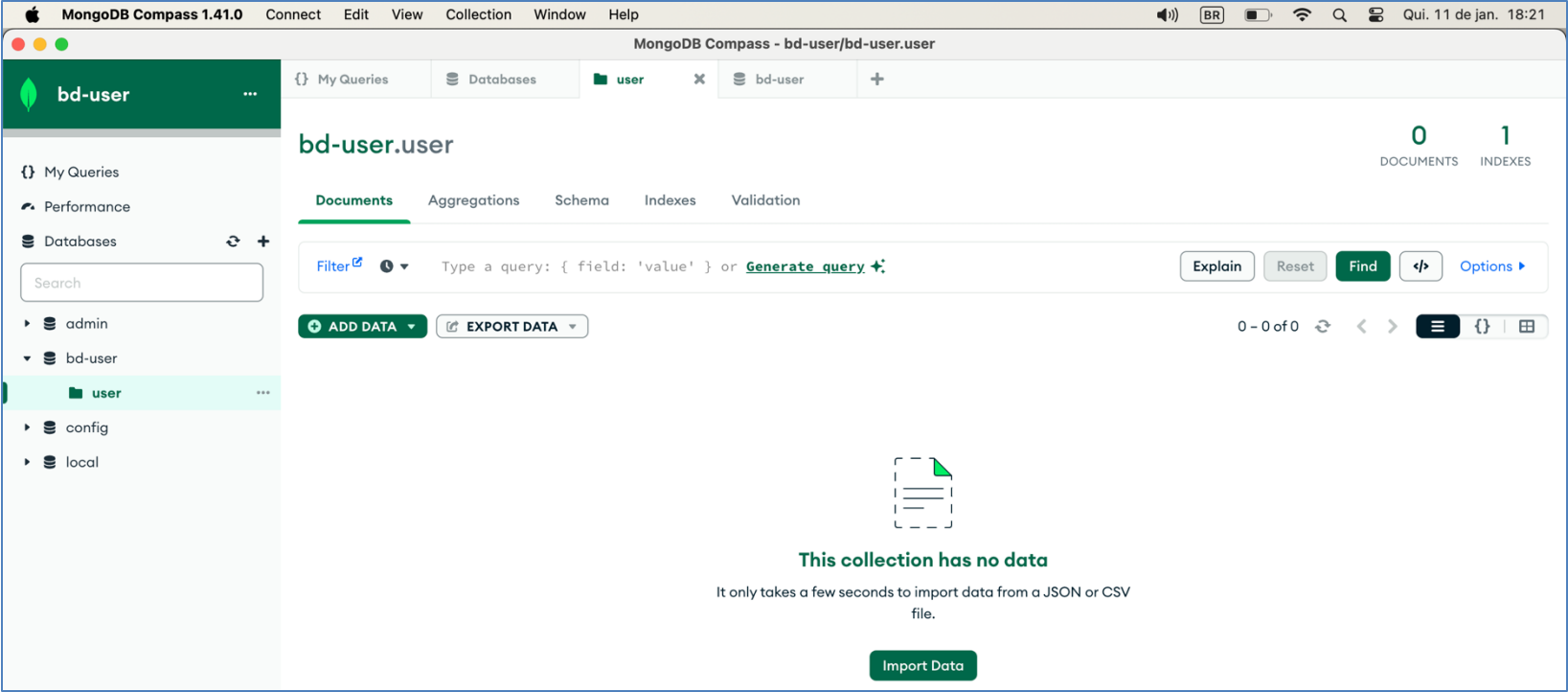
- Gravação e listagem de dados
 - Crie o banco de dados **bd-user** clicando em **+ Create database**.

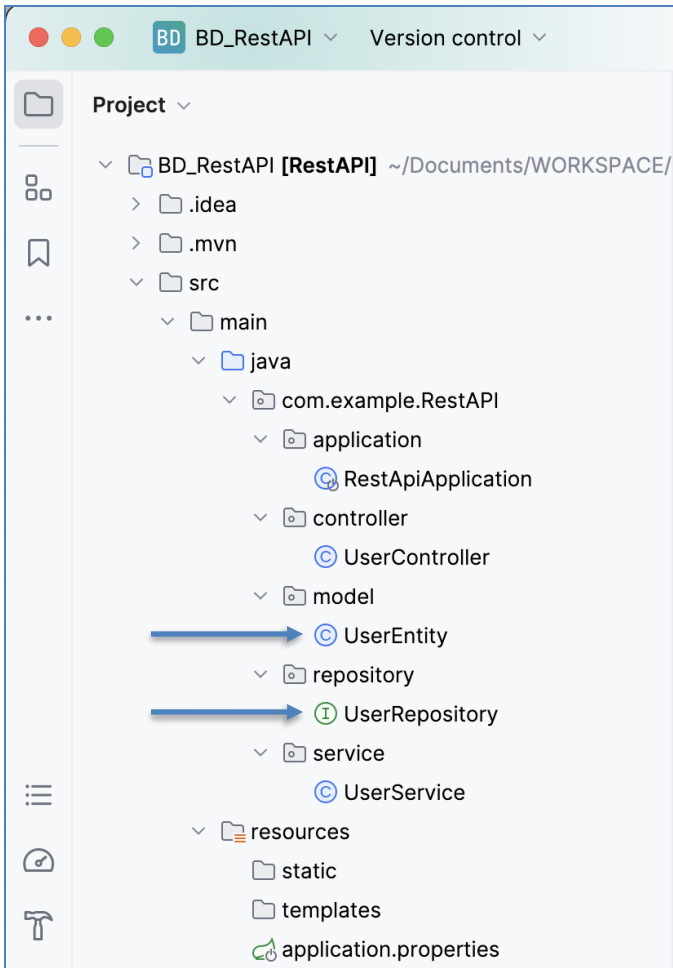


- Gravação e listagem de dados
 - A coleção de documentos (semelhante à uma tabela), se chamará **user**.



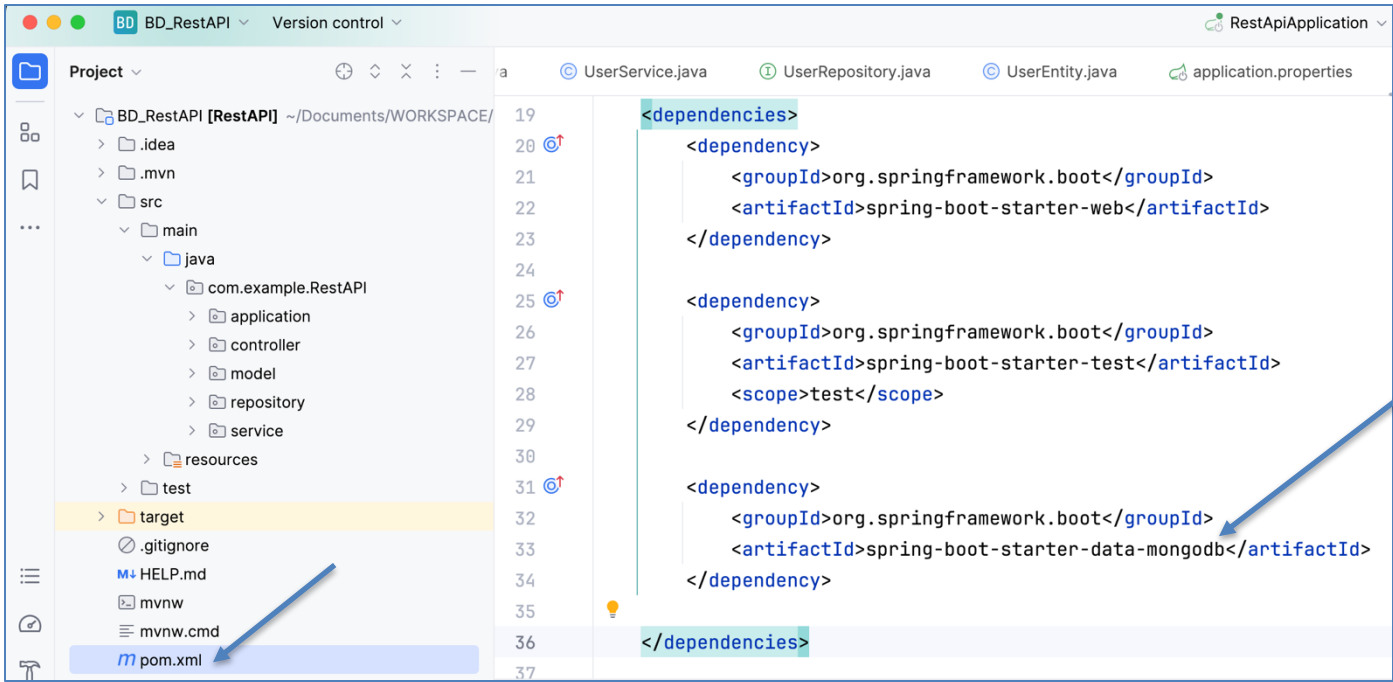
- Gravação e listagem de dados
 - Este será o resultado da criação do banco de dados e da coleção:



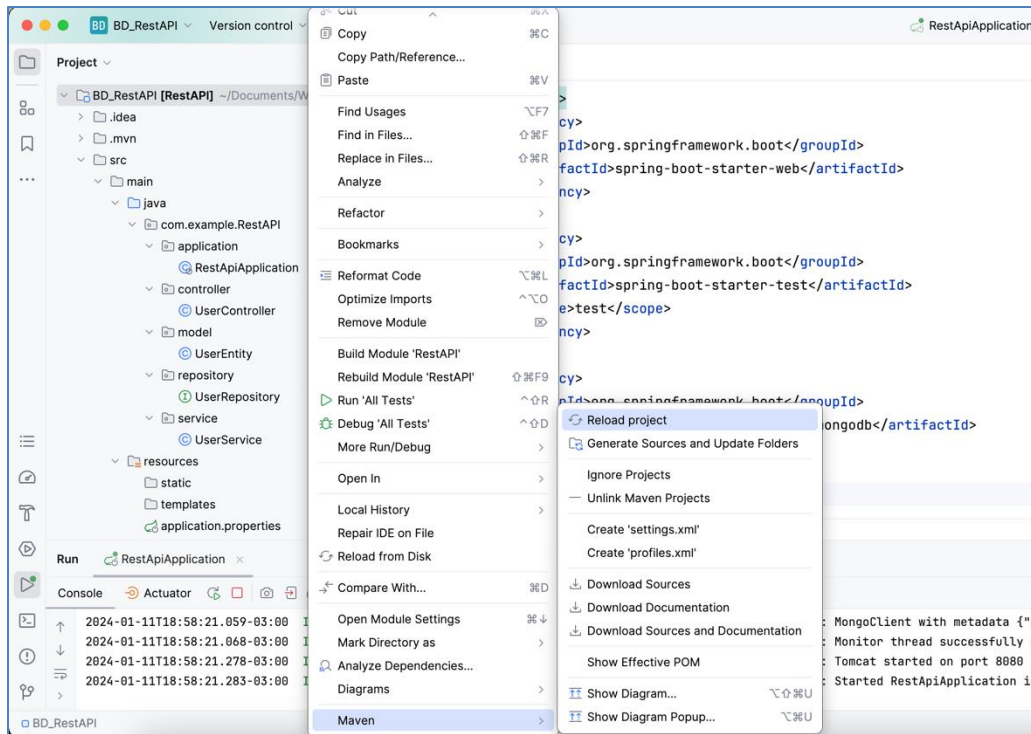


- Veja o que muda na estrutura do projeto:
 - Criamos um pacote novo chamado **model**, com uma classe **UserEntity**.
 - Esta classe representa uma entidade do mundo real, no caso um **usuário**.
 - Criamos outro pacote novo chamado **repository**, com uma interface **UserRepository**.
 - Esta interface será responsável pela interação com o banco de dados.

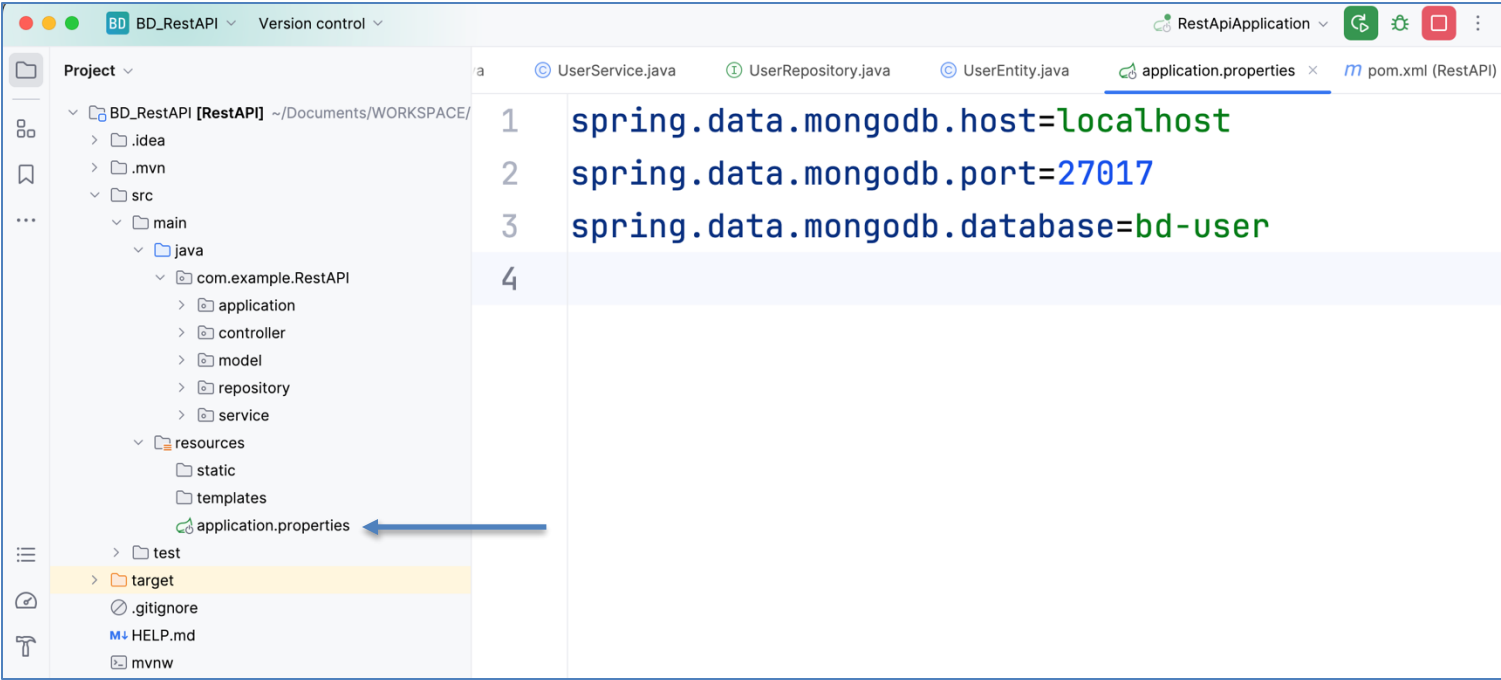
- Gravação e listagem de dados
 - Para utilizarmos o MongoDB na nossa aplicação, precisamos fazer duas configurações:



- **Gravação e listagem de dados**
 - Para atualizar as dependências do projeto, será necessário clicar com o botão direito no projeto e depois ir em: **maven -> Reload project**:



- **Gravação e listagem de dados**
 - Para utilizarmos o MongoDB na nossa aplicação, precisamos fazer duas configurações:



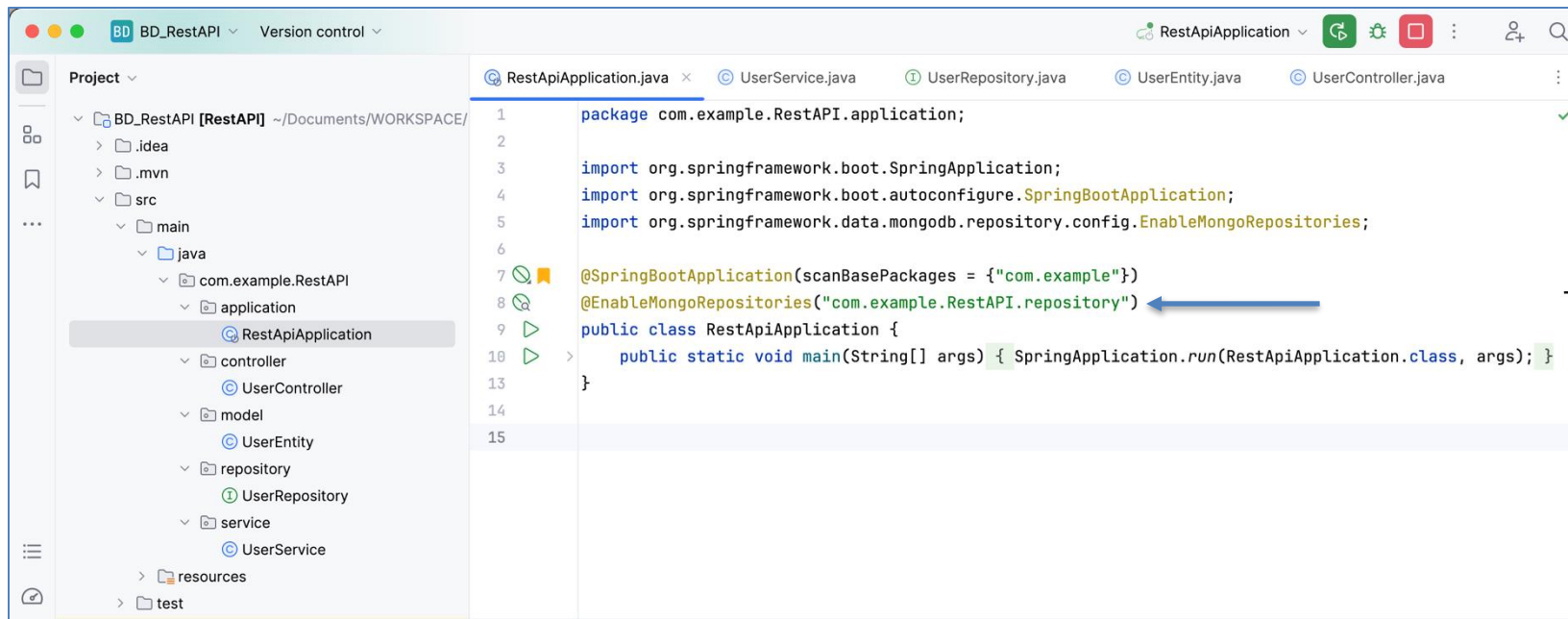
Adicionar o host,
porta e nome do
database

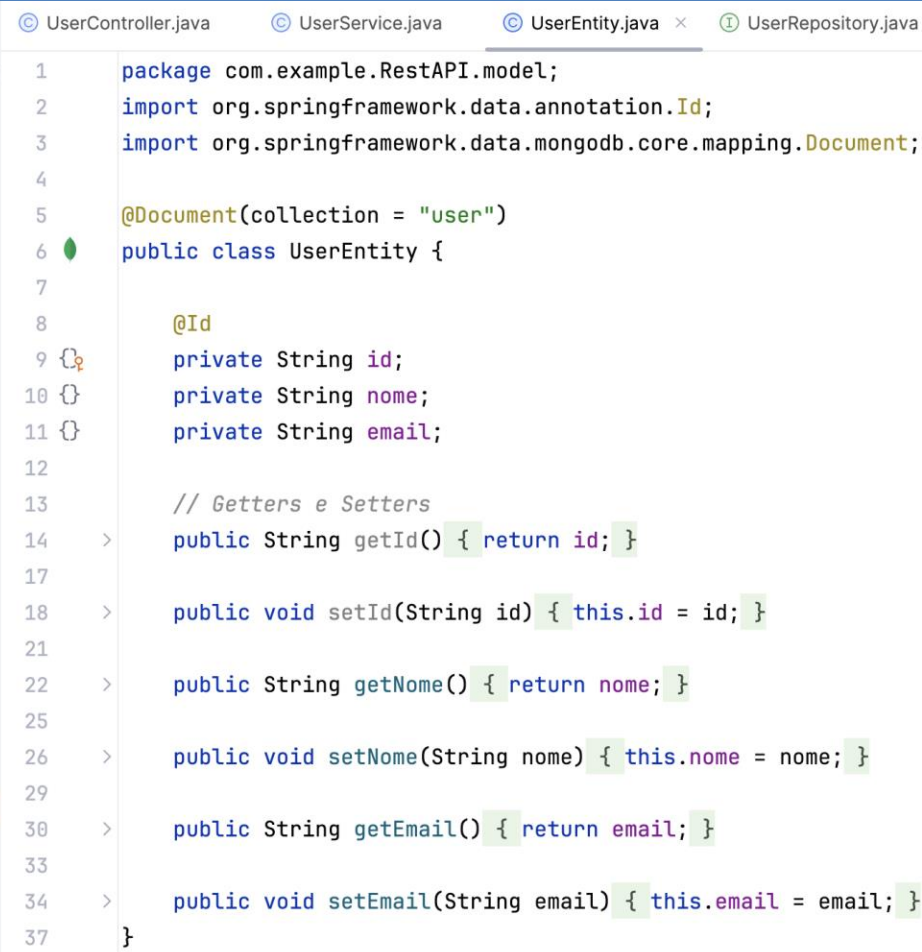
- **localhost** – BD local
- **27017** – porta padrão para o MongoDB
- **bd-user** – nome que demos ao BD

Arquivo
application.properties

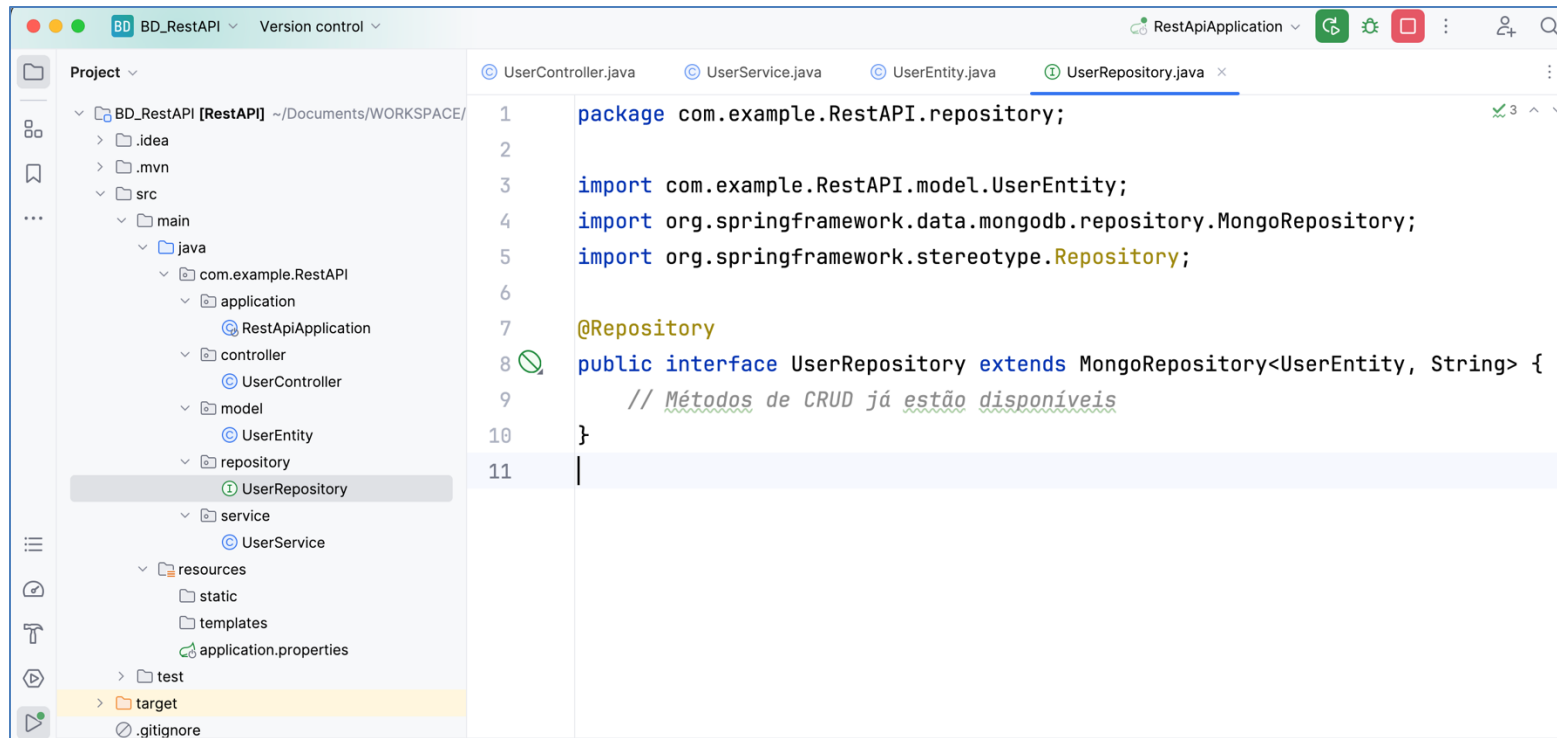
- **Gravação e listagem de dados**

- Classe **RestApiApplication** - O que muda aqui é a utilização da annotation:
 - **@EnableMongoRepositories**("com.example.RestAPI.repository")
 - O argumento dessa anotação especifica o pacote base onde o Spring deve procurar por interfaces de repositório.





- **Gravação e listagem de dados**
 - Interface **UserRepository**: Responsável pela interação com o BD.
 - UserRepository herda comportamentos e métodos de **MongoRepository**.
 - Exemplos: métodos findAll(), findById(), save() e deleteById().



Classe **UserController** com os **endpoints** e as chamadas para a classe **UserService**.

[RestAPI] ~/Documents/WORKSPACE/

com.example.RestAPI

application

RestApiApplication

controller

UserController

model

UserController

repository

UserRepository

service

UserService

sources

static

templates

application.properties

UserController.java

UserService.java

UserEntity.java

UserRepository.java

```
1 package com.example.RestAPI.controller;
2 import com.example.RestAPI.model.UserEntity;
3 import com.example.RestAPI.service.UserService;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.web.bind.annotation.*;
6 import java.util.List;
7
8 @RestController
9 @RequestMapping("/users")
10 public class UserController {
11     @Autowired
12     private UserService userService;
13
14     @GetMapping
15     public List<UserEntity> obterTodos() { return userService.obterTodos(); }
16
17
18
19     @GetMapping("/{id}")
20     public UserEntity obterPorId(@PathVariable String id) { return userService.obterPorId(id); }
21
22
23
24     @PostMapping
25     public UserEntity inserir(@RequestBody UserEntity user) { return userService.inserir(user); }
26
27
28
29     @PutMapping("/{id}")
30     public UserEntity atualizar(@PathVariable String id, @RequestBody UserEntity user) {
31         return userService.atualizar(id, user);
32     }
33
34     @DeleteMapping("/{id}")
35     public void excluir(@PathVariable String id) { userService.excluir(id); }
```

Annotation usada para realizar a **injeção de dependência** (padrão de projeto que permite que o Spring resolva e forneça automaticamente as dependências necessárias para as classes gerenciadas por ele).

- **Gravação e listagem de dados**
 - Classe **UserService**: Contém os serviços de CRUD.
 - Faz a interação com a interface **UserRepository**.

The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a package `com.example.RestAPI` with sub-packages `application`, `controller`, `model`, `repository`, and `service`. The `UserService` class is highlighted in the `service` package. The code editor shows the following code:

```
1 package com.example.RestAPI.service;
2 import com.example.RestAPI.repository.UserRepository;
3 import com.example.RestAPI.model.UserEntity;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.stereotype.Service;
6 import java.util.List;
7
8 @Service
9 public class UserService {
10
11     @Autowired
12     private UserRepository userRepository;
13
14     > public List<UserEntity> obterTodos() { return userRepository.findAll(); }
17
18     > public UserEntity obterPorId(String id) { return userRepository.findById(id).orElse( other: null); }
21
22     > public UserEntity inserir(UserEntity user) { return userRepository.save(user); }
25
26     > public UserEntity atualizar(String id, UserEntity newUser) {...}
40
41     > public void excluir(String id) { userRepository.deleteById(id); }
44 }
```

A callout box with a blue border and a blue arrow pointing to the `@Service` annotation on line 8 contains the text: "Precisamos anotar a classe de serviço com **@Service**".

- **Gravação e listagem de dados**
 - Classe **UserService**: Contém os serviços de CRUD.
 - Faz a interação com a interface **UserRepository**.

The screenshot shows an IDE with a project named 'BD_RestAPI [RestAPI]' and a file named 'UserService.java' open. The code in 'UserService.java' implements the 'UserService' interface, interacting with the 'UserRepository' interface. The methods shown are:

```
package com.example.RestAPI.service;
import com.example.RestAPI.repository.UserRepository;
import com.example.RestAPI.model.UserEntity;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<UserEntity> obterTodos() { return userRepository.findAll(); }

    public UserEntity obterPorId(String id) { return userRepository.findById(id).orElse( other: null); }

    public UserEntity inserir(UserEntity user) { return userRepository.save(user); }

    public UserEntity atualizar(String id, UserEntity newUser) {...}

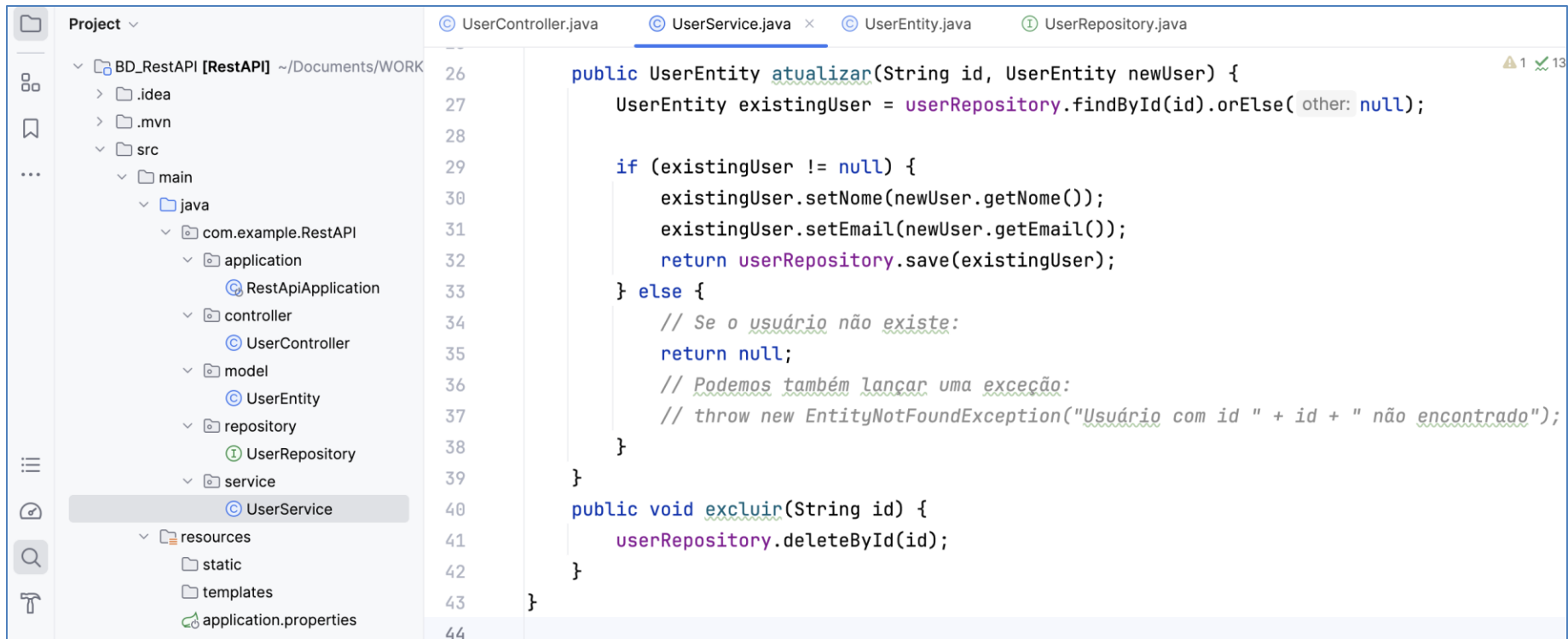
    public void excluir(String id) { userRepository.deleteById(id); }
}
```

A callout box with the text "Gravação e listagem de dados" points to the `obterTodos()` and `obterPorId()` methods, which are responsible for data retrieval (listagem).

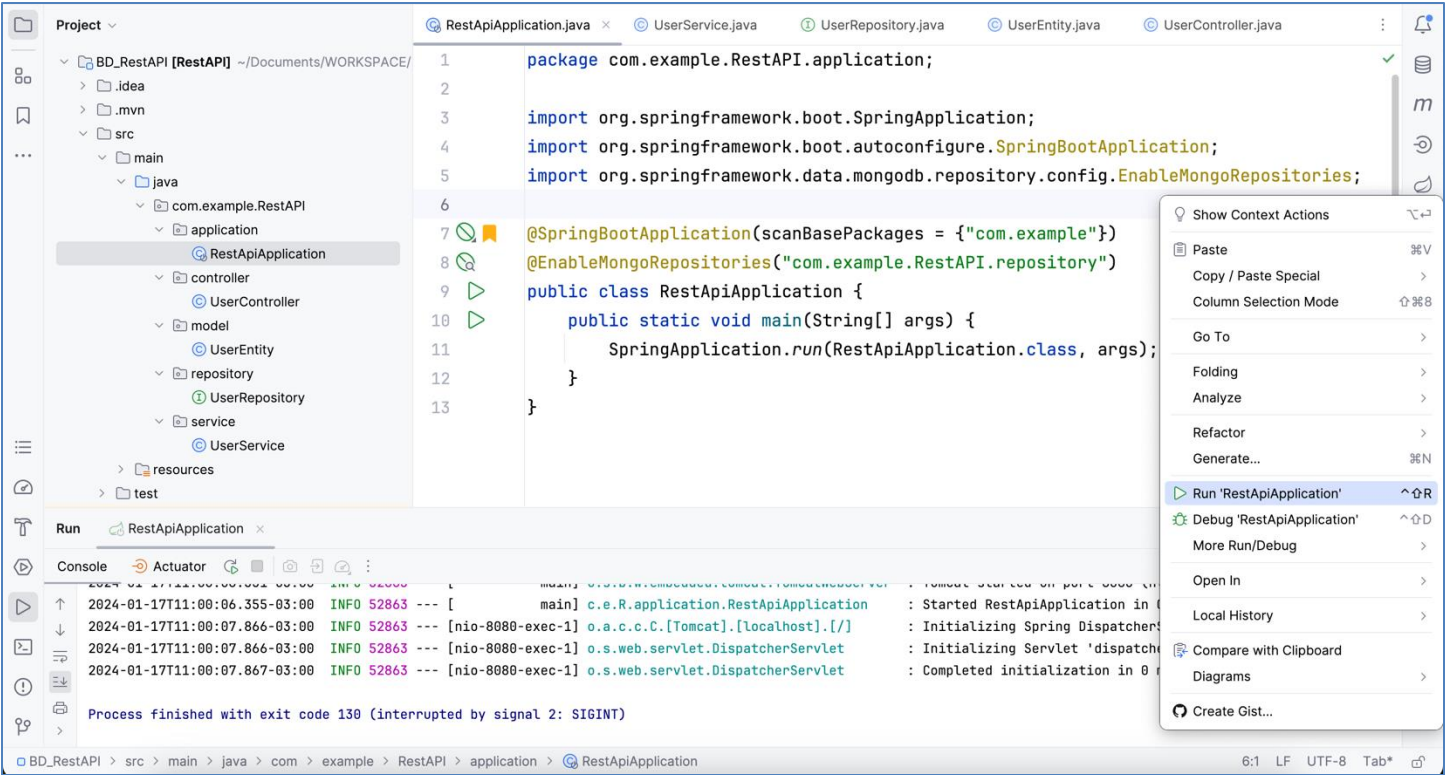
Sumário

- Persistência de dados
 - Banco de dados NoSQL
 - Gravação e listagem de dados
 - **Atualização e exclusão de dados**

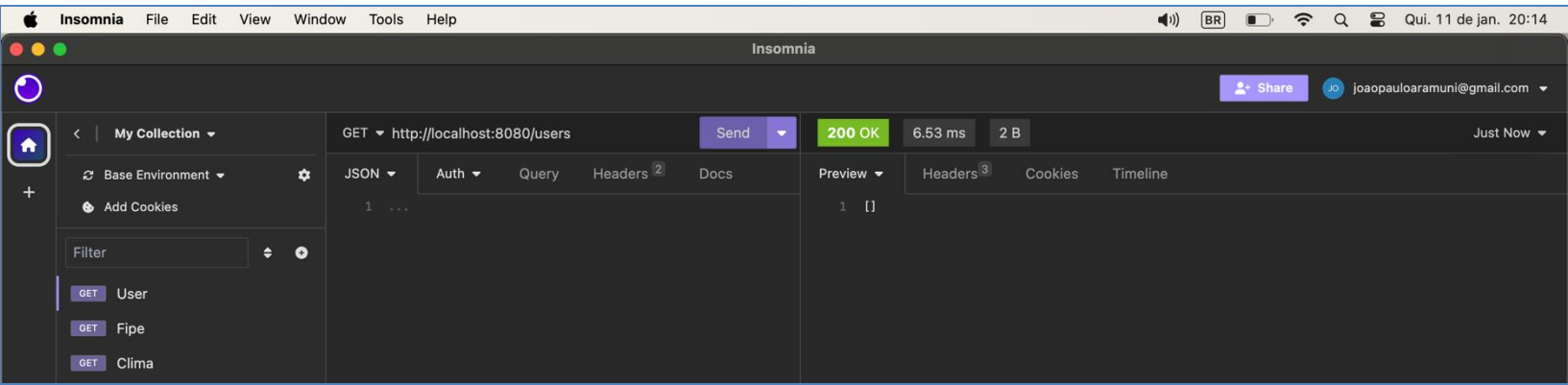
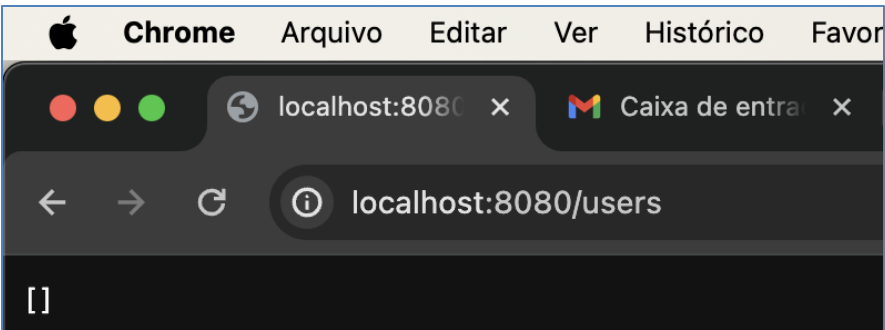
- **Atualização e exclusão de dados**
 - Classe **UserService**: Contém os serviços de CRUD.
 - Faz a interação com a interface **UserRepository**.



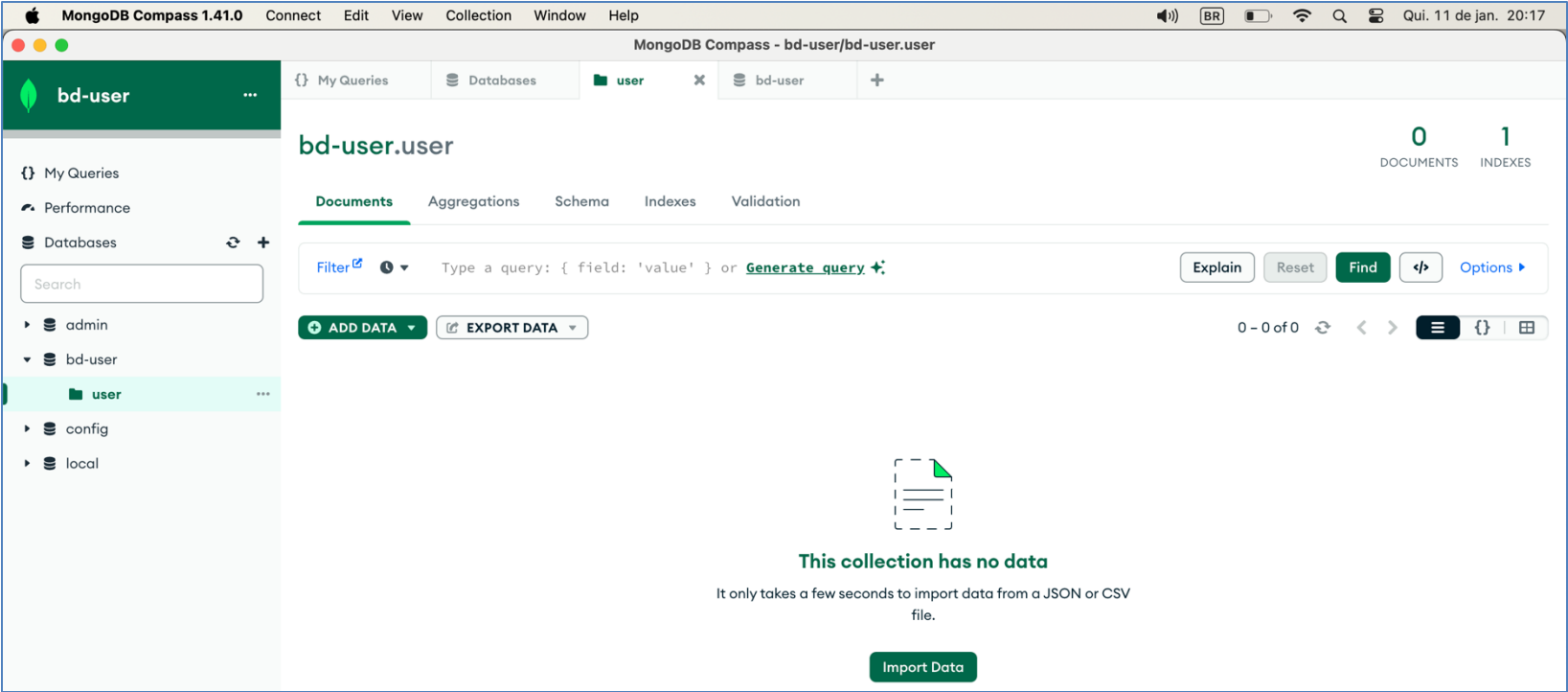
- **Testando a aplicação:**



- **Testando a aplicação:**
 - Por enquanto não temos nenhum usuário cadastrado:

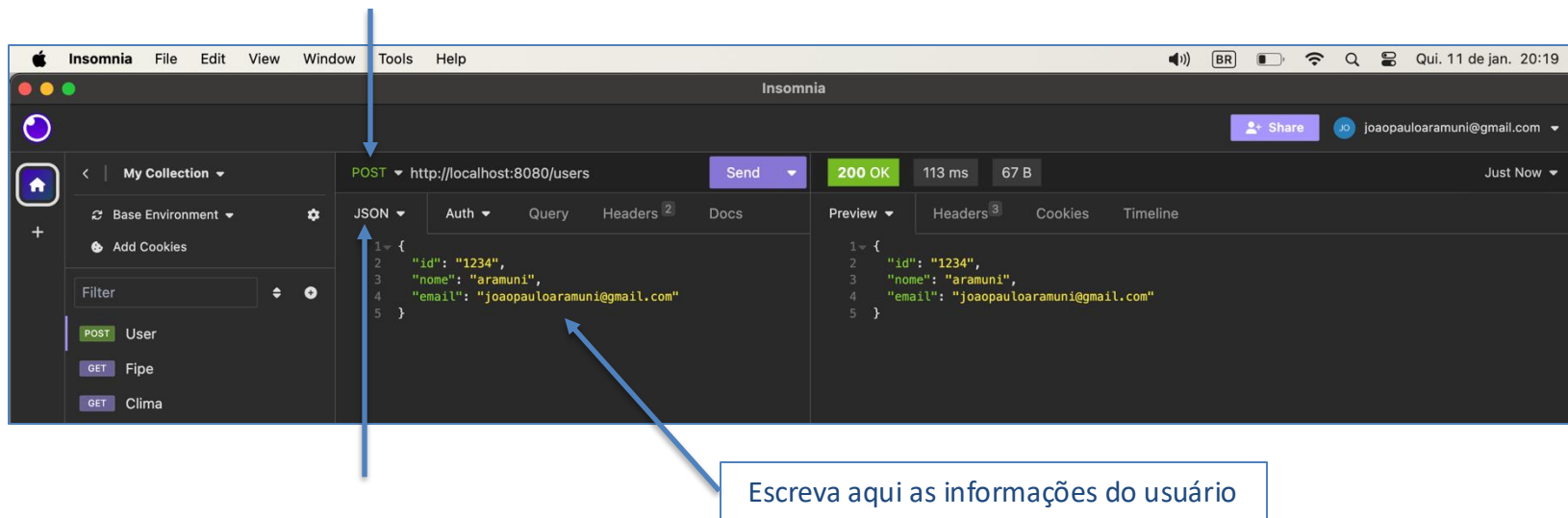


- **Testando a aplicação:**
 - Por enquanto não temos nenhum usuário cadastrado:

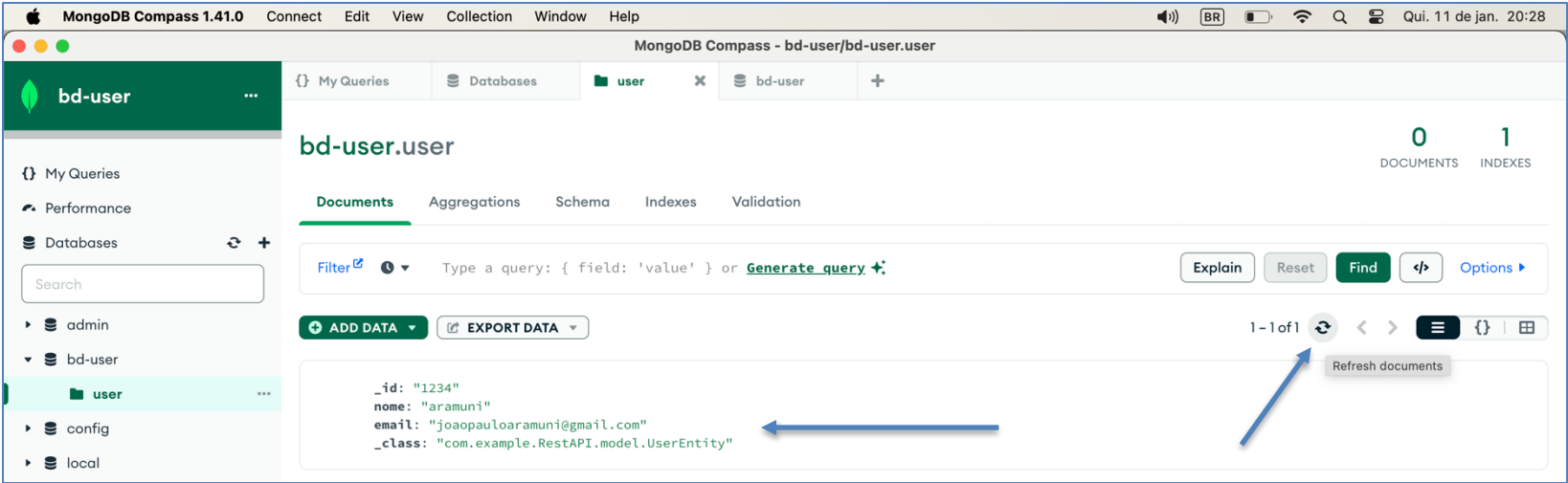
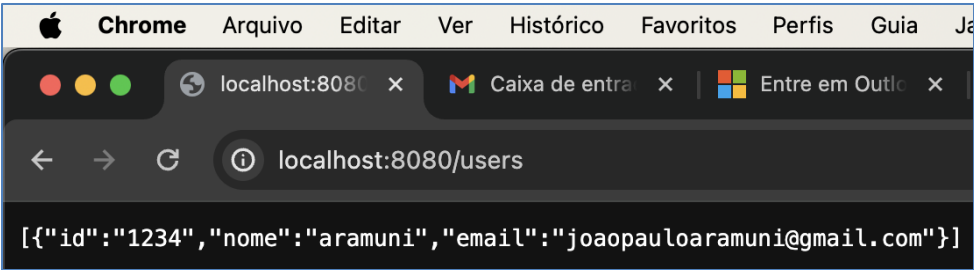


- **Testando a aplicação:**

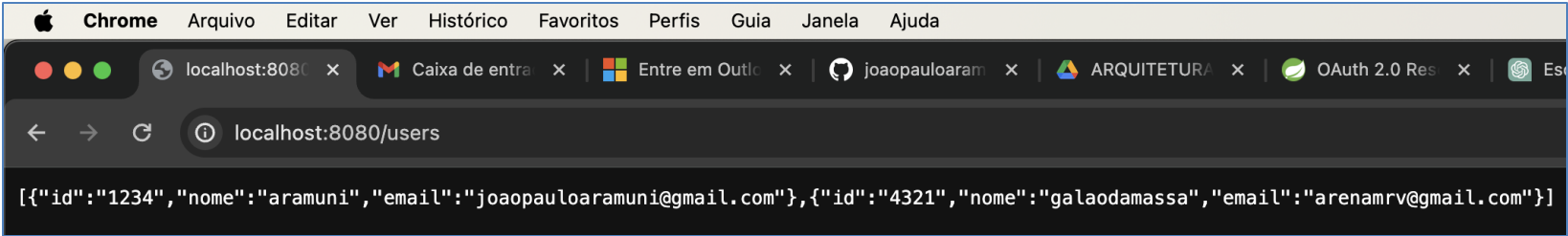
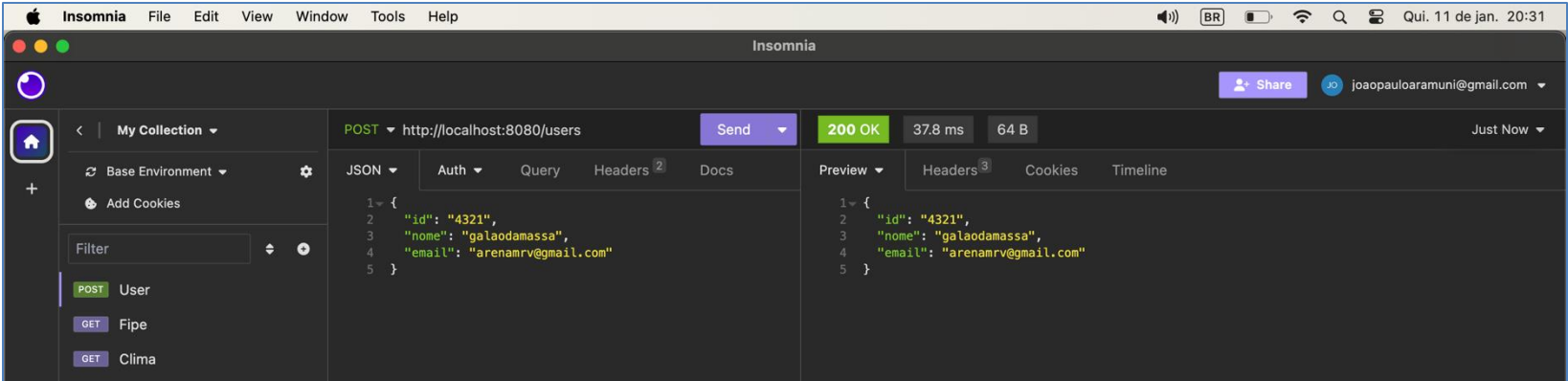
- Para incluir um usuário, precisamos enviar uma requisição HTTP do tipo **POST**.
- Também precisamos escrever o corpo da requisição (@RequestBody) no formato **JSON**. Para isso utilize o **Postman** ou o **Insomnia**.



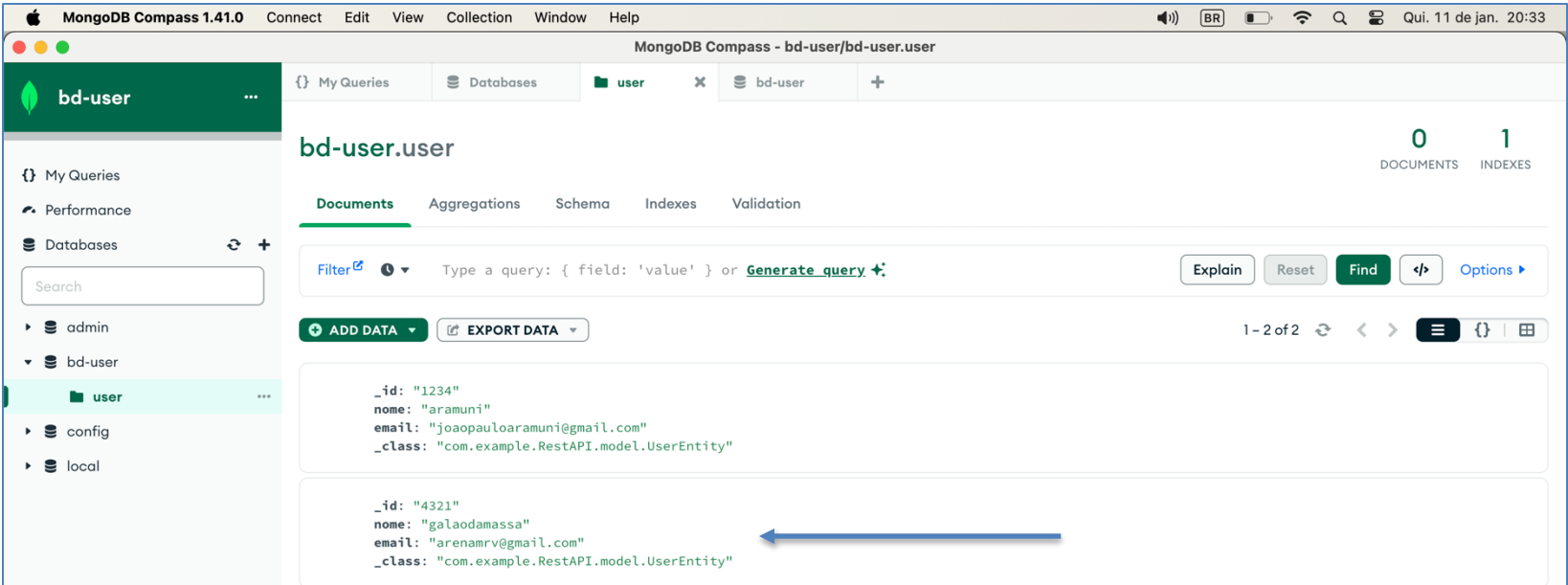
- **Testando a aplicação:**
 - Confira se o usuário foi inserido corretamente:



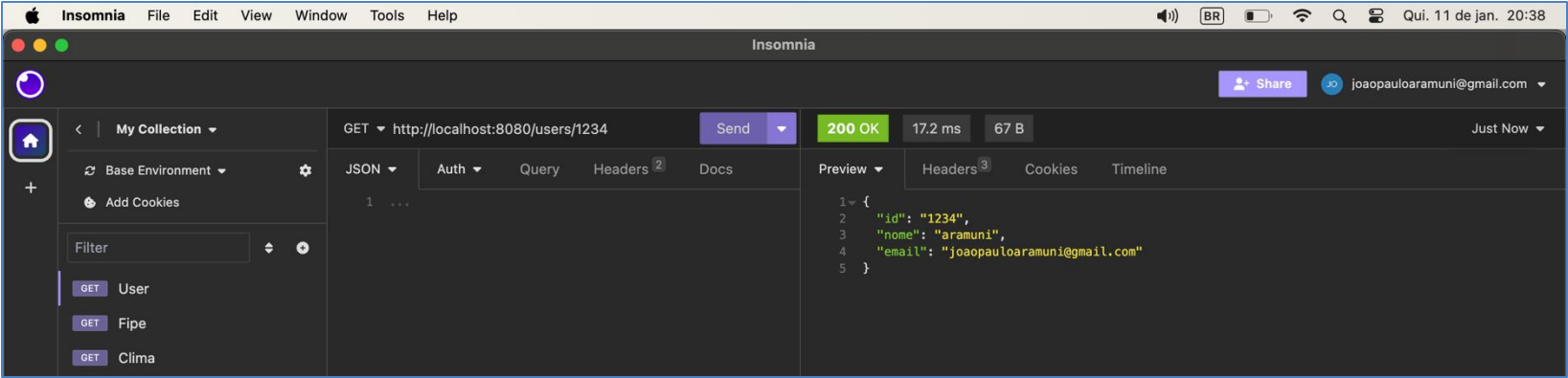
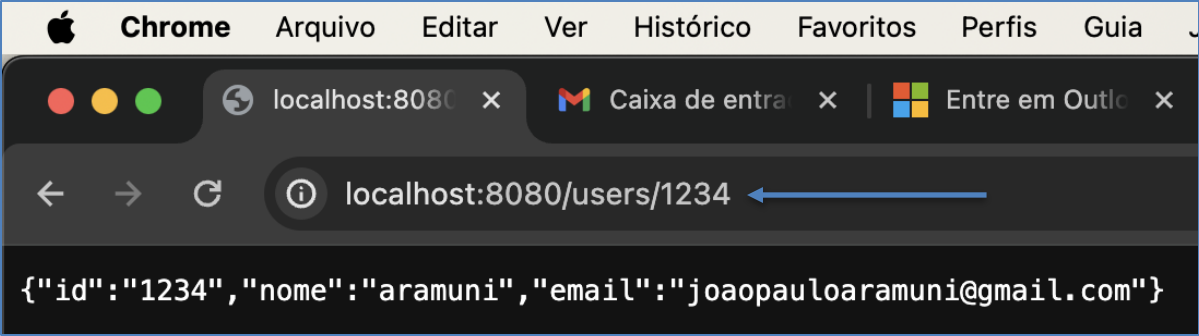
- **Testando a aplicação:**
 - Insira mais um usuário.
 - Assim poderemos saber se a busca por Id está funcionando.



- **Testando a aplicação:**
 - Insira mais um usuário.
 - Assim poderemos saber se a busca por Id está funcionando.



- Testando a aplicação:
 - Listagem por Id:



- **Testando a aplicação:**

- Agora que já testamos os métodos de obterTodos() /users, obterPorId() /users/id, e inserir /users (post), vamos testar os métodos de atualizar e excluir.

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<UserEntity> obterTodos() { return userRepository.findAll(); }

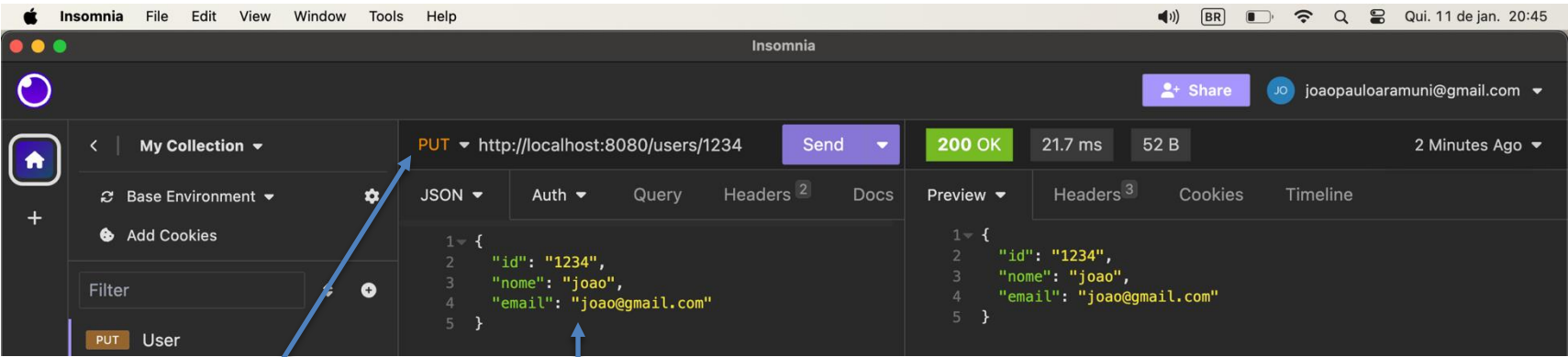
    public UserEntity obterPorId(String id) { return userRepository.findById(id).orElse( other: null); }

    public UserEntity inserir(UserEntity user) { return userRepository.save(user); }
```

- **Testando a aplicação:**

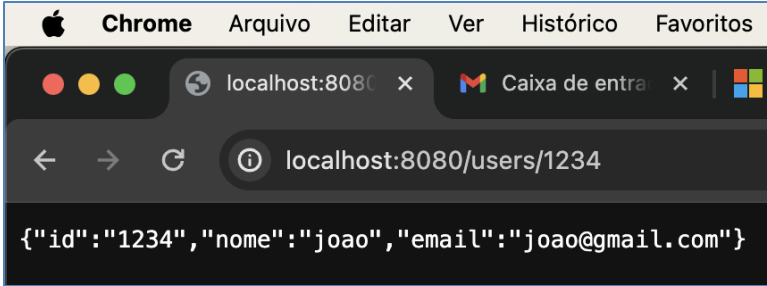
- Atualizar por Id:

Atualizando o nome aramuni para joao e o email para joao@gmail.com.

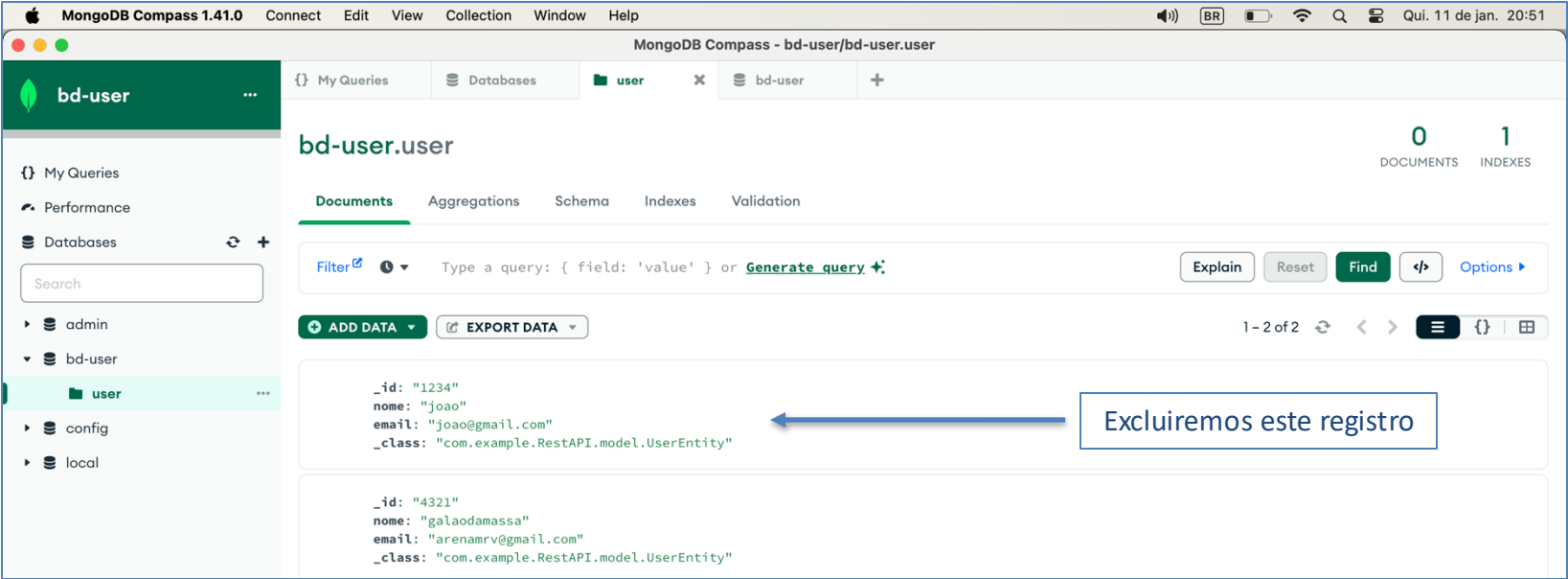


Método PUT

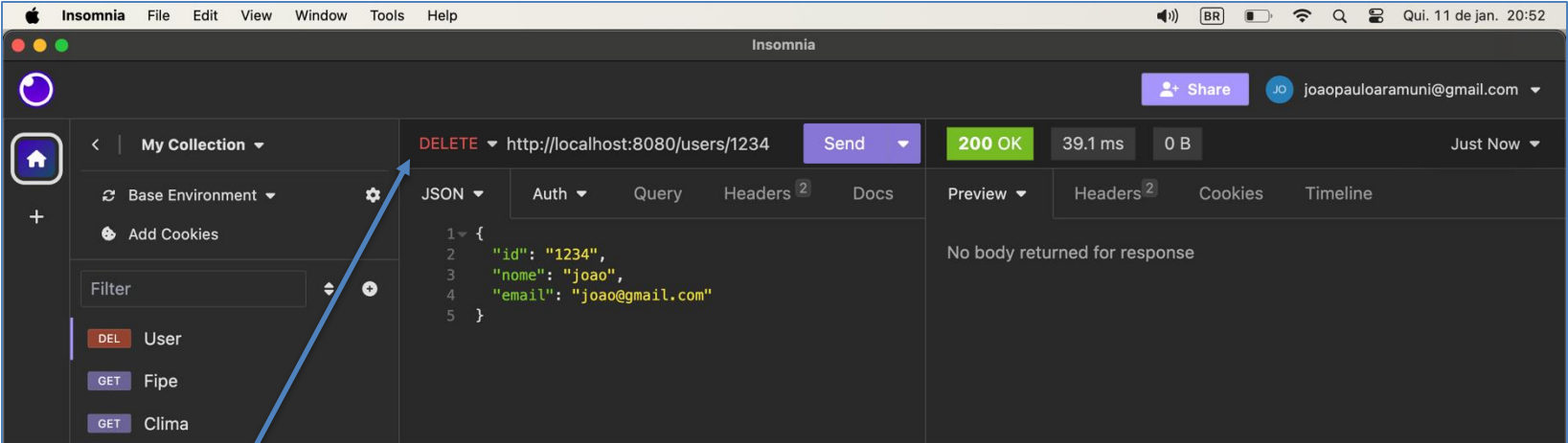
Novas informações



- Testando a aplicação:
 - Excluir por Id:

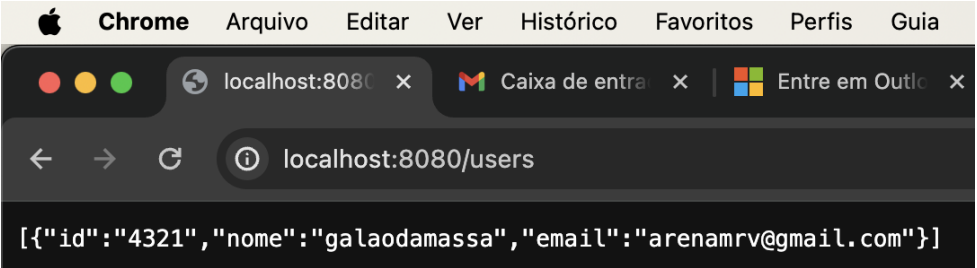


- Testando a aplicação:
 - Excluir por Id:

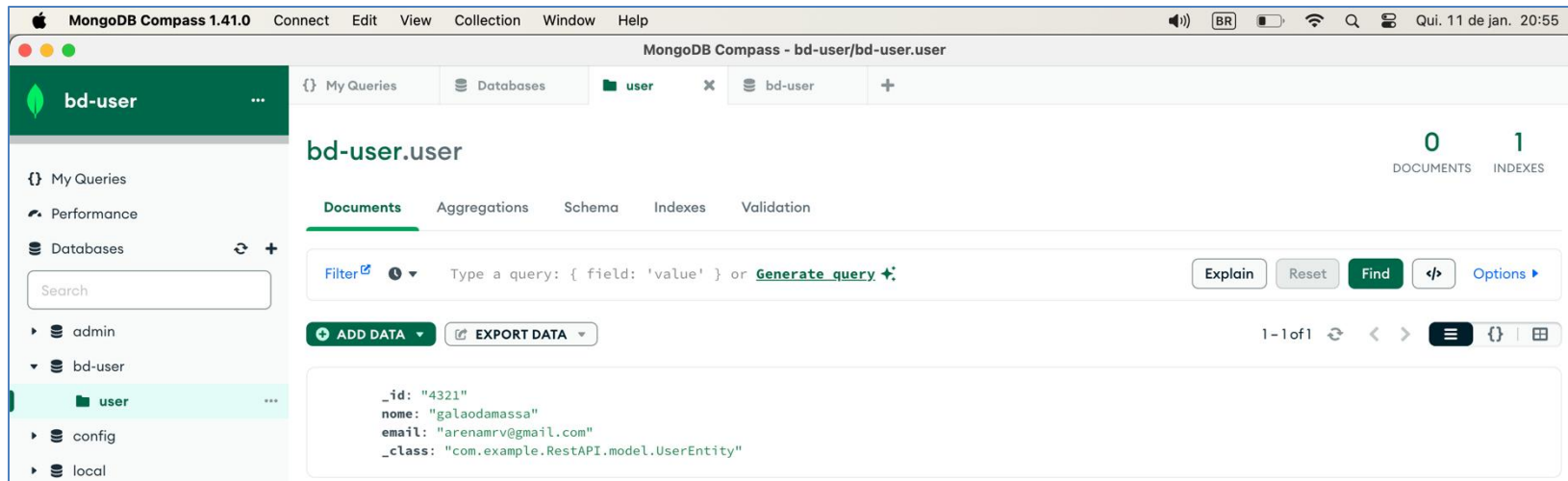


Método **DELETE**

Sobrou apenas este registro



- Testando a aplicação:
 - Excluir por Id:
 - Excluído com sucesso.

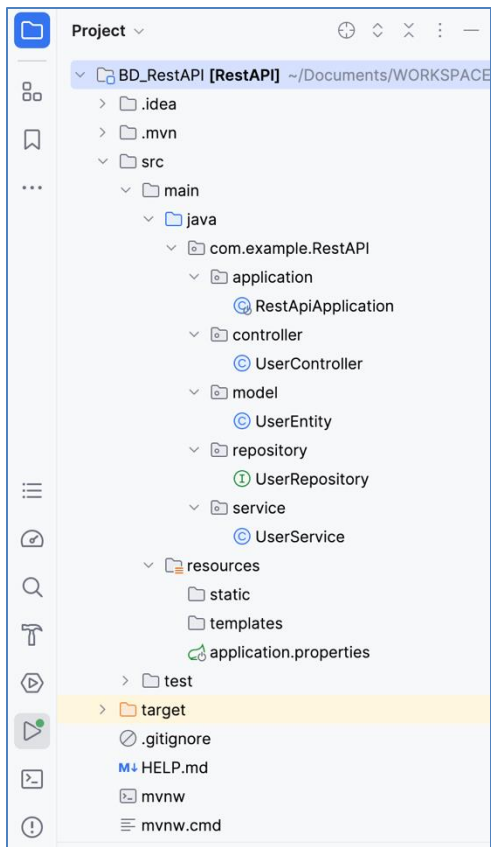


Persistência de dados

- Camada repository

- Conseguimos incluir mais uma camada à nossa arquitetura, a camada **repository**.
- Repare como a arquitetura do projeto vai ganhando mais robustez a cada separação de responsabilidade.
- Essa organização facilita muito a **manutenção**, a **escalabilidade** e a compreensão do código.
- A estrutura do projeto torna-se mais **modular** e **flexível**, facilitando a adição, remoção ou modificação de funcionalidades sem afetar outras partes do código.

Persistência de dados



- **Camada repository**

- A ideia é o desenvolvedor bater o olho nesta estrutura e já saber exatamente onde precisa ir.
- É assim que melhoramos a legibilidade do código e facilitamos a escalabilidade do sistema, trabalhando na arquitetura e nos padrões de projeto.

Persistência de dados

- **MongoDB via terminal**
 - Até o momento manipulamos o nosso banco de dados via interface gráfica, utilizando o **MongoDB Compass**.
 - Vejamos agora como manipular o MongoDB via **terminal**.

Persistência de dados

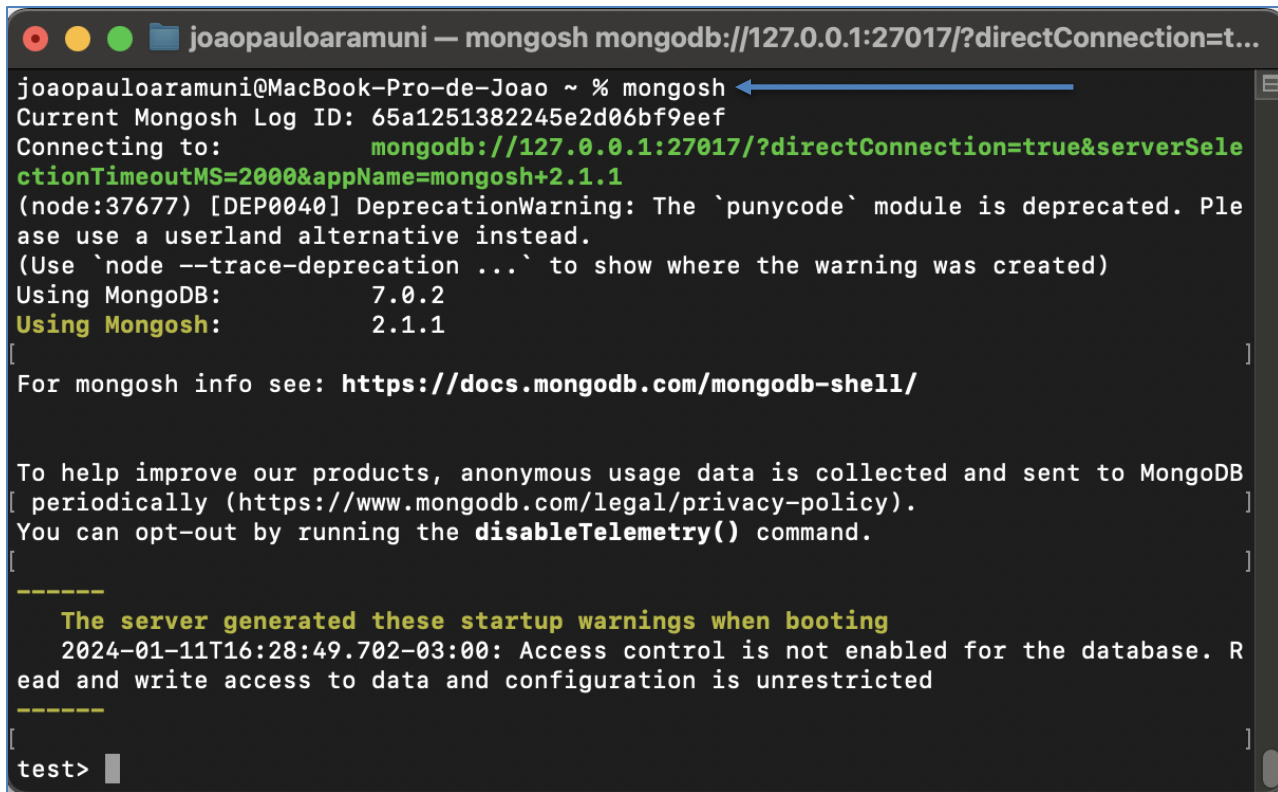
- **MongoDB via terminal**
 - Para instalar o MongoDB e utilizá-lo via terminal, siga o tutorial de acordo com o seu sistema operacional:
 - <https://www.mongodb.com/docs/manual/tutorial/>
 - <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/>
 - <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-red-hat/>
 - <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-ubuntu/>
 - <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-os-x/>

Persistência de dados

- MongoDB via terminal
 - Para startar a sessão do MongoDB, utilizamos o comando **mongosh**.
 - <https://www.mongodb.com/docs/mongodb-shell/#mongodb-binary-bin.mongosh>
 - É exatamente neste terminal que iremos executar os comandos do MongoDB para inserir, listar, atualizar e deletar documentos.
 - <https://www.mongodb.com/docs/manual/tutorial/insert-documents/>
 - <https://www.mongodb.com/docs/manual/tutorial/query-documents/>
 - <https://www.mongodb.com/docs/manual/tutorial/update-documents/>
 - <https://www.mongodb.com/docs/manual/tutorial/remove-documents/>

- **MongoDB via terminal**

- mongosh



```
joaopauloaramuni — mongosh mongodb://127.0.0.1:27017/?directConnection=t...
joaopauloaramuni@MacBook-Pro-de-Joao ~ % mongosh
Current Mongosh Log ID: 65a1251382245e2d06bf9eef
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.1.1
(node:37677) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
Using MongoDB:      7.0.2
Using Mongosh:      2.1.1

[
  For mongosh info see: https://docs.mongodb.com/mongosh-shell/
]

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

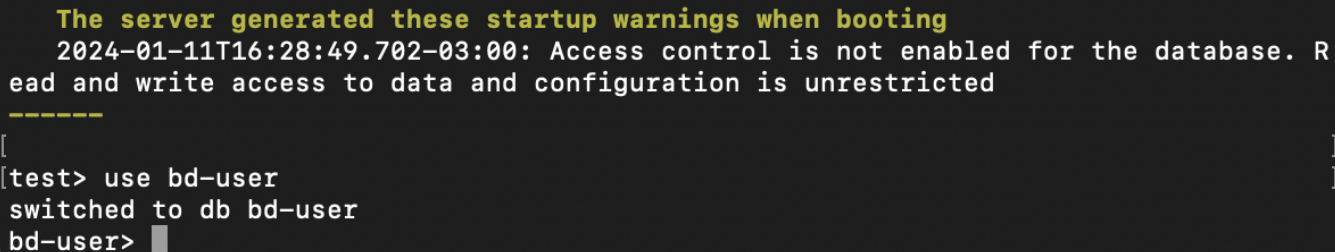
[
  -----
  The server generated these startup warnings when booting
  2024-01-11T16:28:49.702-03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  -----
]

test> 
```

Persistência de dados

- Primeiramente, precisamos selecionar um banco de dados utilizando o comando **use**.

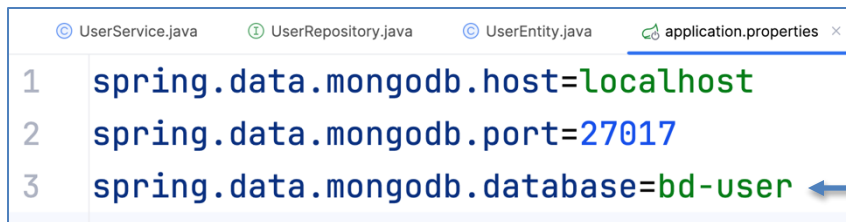
Este mesmo comando também serve para criar um novo banco de dados.



The server generated these startup warnings when booting
2024-01-11T16:28:49.702-03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted

[test> use bd-user
switched to db bd-user
bd-user>]

A blue arrow points from the left towards the terminal window.



UserService.java UserRepository.java UserEntity.java application.properties x

```
1 spring.data.mongodb.host=localhost
2 spring.data.mongodb.port=27017
3 spring.data.mongodb.database=bd-user
```

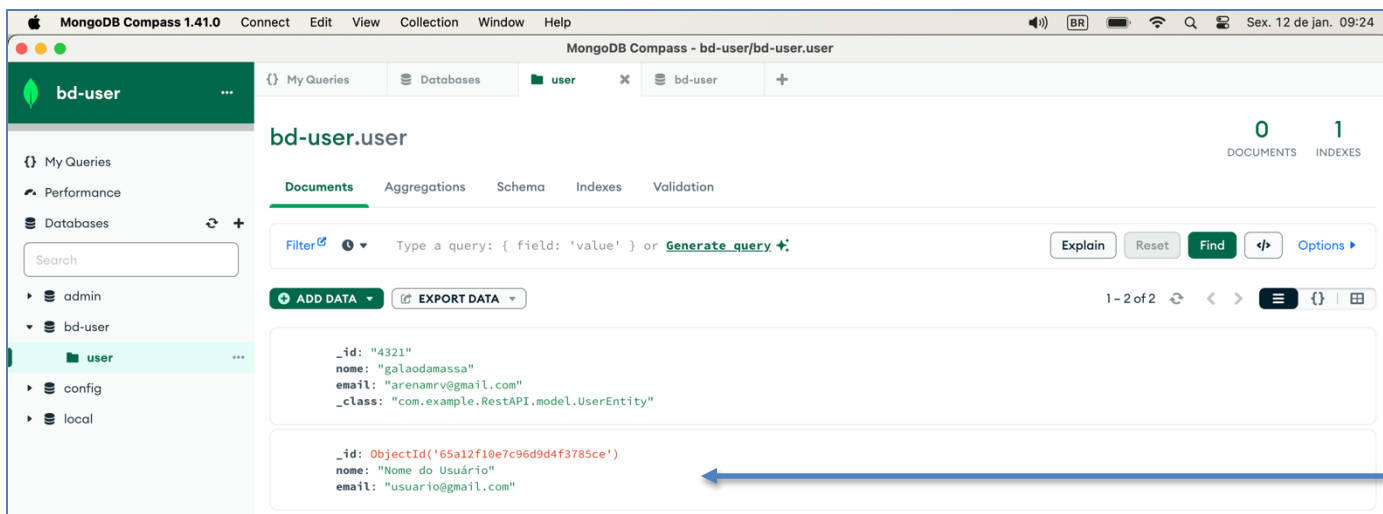
A blue arrow points from the right towards the value 'bd-user' in the third line.

Persistência de dados

- Agora podemos inserir um novo documento à coleção **user** (o MongoDB criará automaticamente essa coleção ao inserirmos o primeiro registro).
- Para isso utilizamos o método **insertOne()** ou **insertMany()**.
 - <https://www.mongodb.com/docs/manual/tutorial/insert-documents/>

- Repare que o MongoDB gera o ID automaticamente para você.

```
[test> use bd-user
switched to db bd-user
bd-user> db.user.insertOne({nome: "Nome do Usuário", email: "usuario@gmail.com"})
{
  acknowledged: true,
  insertedId: ObjectId('65a12f10e7c96d9d4f3785ce')
}
bd-user>
```



Confira se o dado foi inserido.
Utilize a interface gráfica do
MongoDB Compass.

Persistência de dados

- Você também pode inserir vários documentos de uma só vez:

```
MongoDB Shell ▼  
  
db.inventory.insertMany([  
  { item: "journal", qty: 25, size: { h: 14, w: 21, uom: "cm" }, status: "A" },  
  { item: "notebook", qty: 50, size: { h: 8.5, w: 11, uom: "in" }, status: "A" },  
  { item: "paper", qty: 100, size: { h: 8.5, w: 11, uom: "in" }, status: "D" },  
  { item: "planner", qty: 75, size: { h: 22.85, w: 30, uom: "cm" }, status: "D" },  
  { item: "postcard", qty: 45, size: { h: 10, w: 15.25, uom: "cm" }, status: "A" }  
]);
```

Persistência de dados

- Vamos agora listar os documentos gravados.
- Para isso utilizamos o método **find()**.
 - <https://www.mongodb.com/docs/manual/tutorial/query-documents/>

Persistência de dados

- Obter todos:

```
joapauloaramuni — mongosh mongodb://127.0.0.1:27017/?directConnectio...  
-----  
[test> use bd-user  
switched to db bd-user  
[bd-user> db.user.insertOne({nome: "Nome do Usuário", email: "usuario@gmail.com"})  
{  
  acknowledged: true,  
  insertedId: ObjectId('65a12f10e7c96d9d4f3785ce')  
}  
[bd-user> db.user.find() ←  
[  
  {  
    _id: '4321',  
    nome: 'galaodamassa',  
    email: 'arenamrv@gmail.com',  
    _class: 'com.example.RestAPI.model.UserEntity'  
  },  
  {  
    _id: ObjectId('65a12f10e7c96d9d4f3785ce'),  
    nome: 'Nome do Usuário',  
    email: 'usuario@gmail.com'  
  }  
]  
bd-user> █
```

Persistência de dados

- Listar com uma condição/filtro:

```
joaopauloaramuni — mongosh mongodb://127.0.0.1:27017/?directConnectio...
[bd-user> db.user.find()
[
  {
    _id: '4321',
    nome: 'galaodamassa',
    email: 'arenamrv@gmail.com',
    _class: 'com.example.RestAPI.model.UserEntity'
  },
  {
    _id: ObjectId('65a12f10e7c96d9d4f3785ce'),
    nome: 'Nome do Usuário',
    email: 'usuario@gmail.com'
  }
]
[bd-user> db.user.find({nome: "galaodamassa"}) ←
[
  {
    _id: '4321',
    nome: 'galaodamassa',
    email: 'arenamrv@gmail.com',
    _class: 'com.example.RestAPI.model.UserEntity'
  }
]
[bd-user> ]
```

Persistência de dados

- Hora de atualizar os dados.
- Para isso utilizamos o método **updateOne()**.
 - <https://www.mongodb.com/docs/manual/tutorial/update-documents/>

Persistência de dados

- Atualizando apenas um documento:

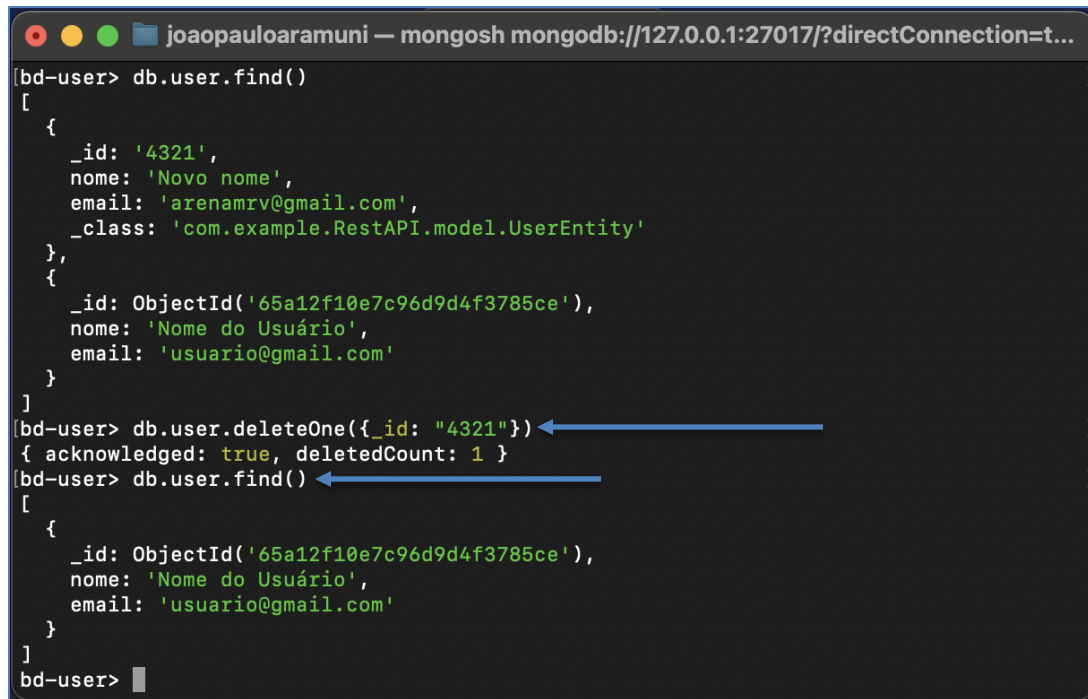
```
joaopauloaramuni — mongosh mongodb://127.0.0.1:27017/?directConnectio...
[bd-user> db.user.updateOne({_id:"4321"}, {$set: {nome: "Novo nome"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[bd-user> db.user.find()
[
  {
    _id: '4321',
    nome: 'Novo nome',
    email: 'arenamrv@gmail.com',
    _class: 'com.example.RestAPI.model.UserEntity'
  },
  {
    _id: ObjectId('65a12f10e7c96d9d4f3785ce'),
    nome: 'Nome do Usuário',
    email: 'usuario@gmail.com'
  }
]
bd-user>
```


Persistência de dados

- Para atualizar vários documentos de uma vez, utilizamos o método **updateMany()**.
 - <https://www.mongodb.com/docs/manual/tutorial/update-documents/>
- Por fim, podemos excluir um documento. Para isso utilizamos os métodos **deleteOne()** e **deleteMany()**.
 - <https://www.mongodb.com/docs/manual/tutorial/remove-documents/>

Persistência de dados

- Removendo apenas um documento:

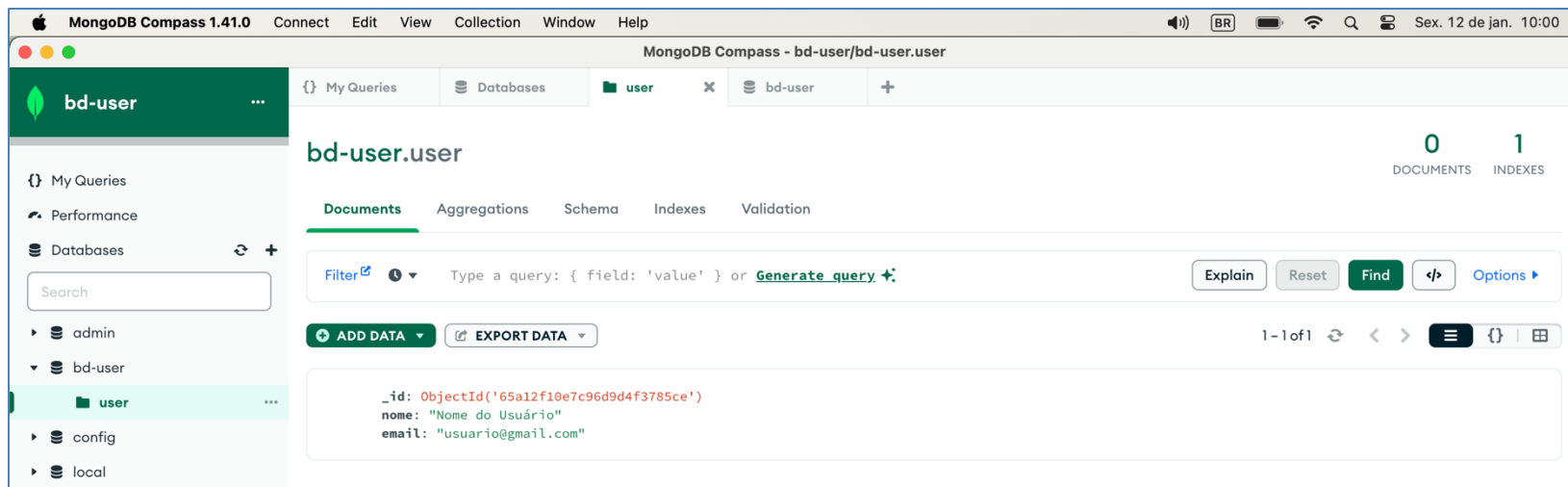


```
joaopauloaramuni — mongosh mongodb://127.0.0.1:27017/?directConnection=t...  
[bd-user> db.user.find()  
[  
  {  
    _id: '4321',  
    nome: 'Novo nome',  
    email: 'arenamrv@gmail.com',  
    _class: 'com.example.RestAPI.model.UserEntity'  
  },  
  {  
    _id: ObjectId('65a12f10e7c96d9d4f3785ce'),  
    nome: 'Nome do Usuário',  
    email: 'usuario@gmail.com'  
  }  
]  
[bd-user> db.user.deleteOne({_id: "4321"})  
{ acknowledged: true, deletedCount: 1 }  
[bd-user> db.user.find()  
[  
  {  
    _id: ObjectId('65a12f10e7c96d9d4f3785ce'),  
    nome: 'Nome do Usuário',  
    email: 'usuario@gmail.com'  
  }  
]  
bd-user> 
```

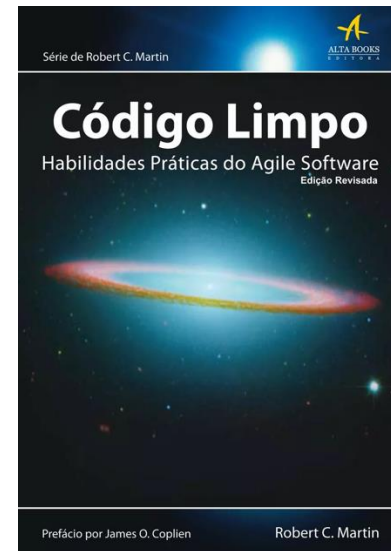
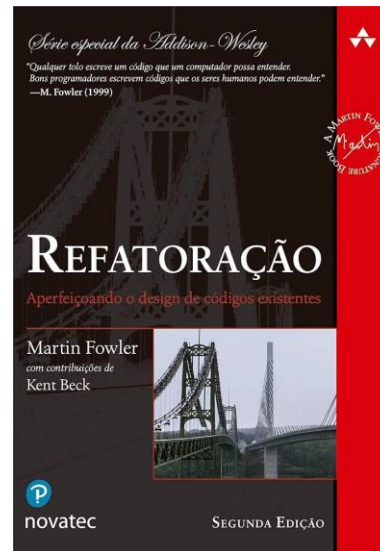
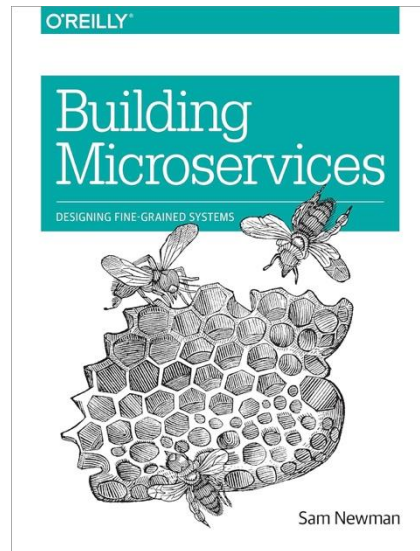
The screenshot shows a MongoDB terminal window. The first command, `db.user.find()`, returns a list of two documents. The second command, `db.user.deleteOne({_id: "4321"})`, successfully deletes the document with `_id: '4321'`. The third command, `db.user.find()`, is run again, and the result shows only the remaining document with `_id: ObjectId('65a12f10e7c96d9d4f3785ce')`. Blue arrows point from the terminal output to the corresponding commands.

Persistência de dados

- Já sabemos que o documento foi deletado.
- Apenas para treinar, confira também na interface gráfica do **MongoDB Compass**.



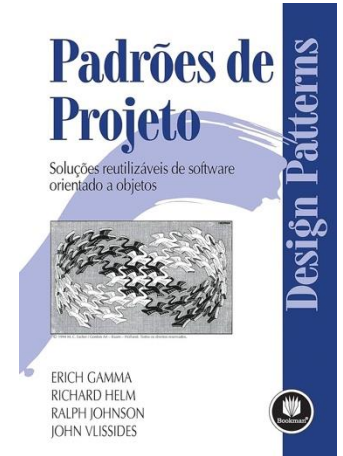
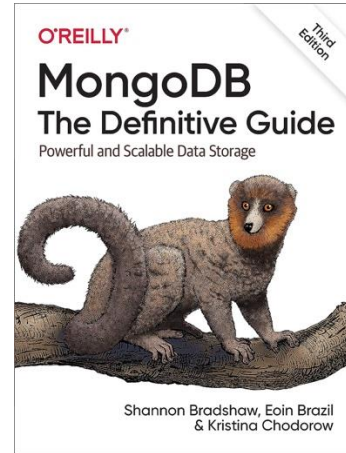
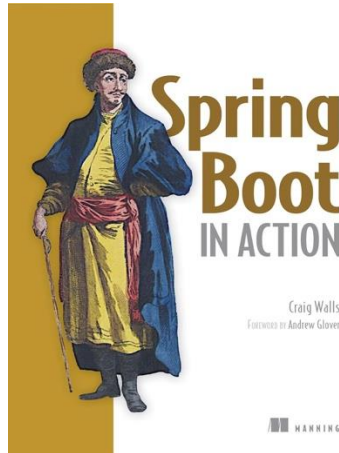
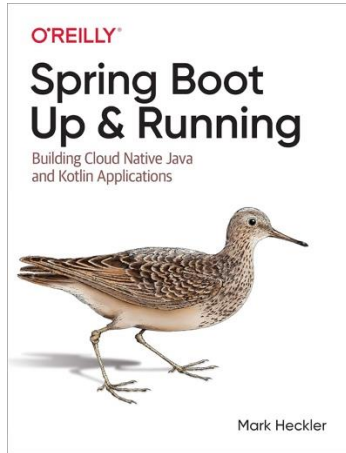
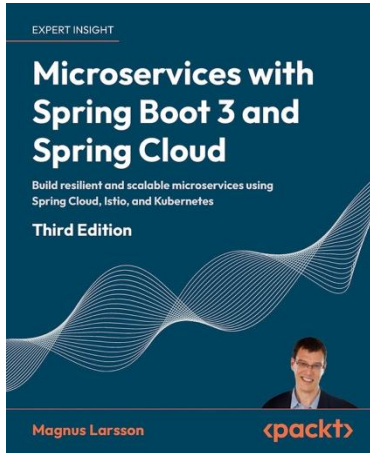
Referências básicas



Referências complementares



Outras referências



Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com



[GitHub](#)



[LinkedIn](#)



[Lattes](#)

...